Building a GPT from Scratch

A six-chapter tutorial on transformers and language modeling

Based on "Attention Is All You Need" (Vaswani et al., 2017),
"The Annotated Transformer" (Rush, Huang et al.),
and Karpathy's nanoGPT

Contents

1 Attention from First Principles **2**
1.1 Project Setup . . . 2
1.2 What Problem Does Attention Solve? . . . 2
1.3 Dot Products as Similarity . . . 3
1.4 Why Scale by $\sqrt{d_k}$ . . . 4
1.5 Softmax Turns Scores into Weights . . . 5
1.6 The Full Attention Formula . . . 6
1.7 Multi-Head Attention . . . 8
1.8 Causal (Autoregressive) Masking . . . 9
1.9 Looking Ahead . . . 10

2 The Transformer Block **12**
2.1 Residual Connections . . . 12
2.1.1 The degradation problem . . . 12
2.1.2 The ResNet fix . . . 12
2.2 Layer Normalization . . . 13
2.2.1 Why not batch normalization? . . . 13
2.2.2 Layer normalization . . . 13
2.2.3 Implementation . . . 14
2.3 Pre-Norm vs Post-Norm . . . 14
2.3.1 Why pre-norm won . . . 15
2.4 Position-wise Feed-Forward Network . . . 15
2.4.1 Architecture . . . 15
2.4.2 Activation function . . . 16
2.4.3 Why "position-wise" . . . 16
2.4.4 Implementation . . . 16
2.5 Assembling the Transformer Block . . . 17
2.5.1 Implementation . . . 17
2.6 Stacking Blocks . . . 17
2.6.1 Parameter count . . . 18
2.7 Looking Ahead . . . 18

3 The Full Transformer (Encoder-Decoder) **19**
3.1 The Sequence-to-Sequence Problem . . . 19
3.2 The Transformer Encoder . . . 20
3.3 The Transformer Decoder . . . 20
3.4 Cross-Attention: How the Decoder Reads the Encoder . . . 21
3.5 Embeddings . . . 22
3.6 Positional Encoding . . . 22
3.7 The Output Head (Generator) . . . 24

3.8 Putting It All Together . . . 25
3.9 Looking Ahead . . . 26

4 From the Transformer to GPT **28**
4.1 Three Transformer Variants . . . 28
4.1.1 Encoder-Decoder (Vaswani et al., 2017) . . . 28
4.1.2 Encoder-Only (BERT, Devlin et al., 2018) . . . 28
4.1.3 Decoder-Only (GPT, Radford et al., 2018) . . . 28
4.2 What GPT Removes . . . 29
4.3 What GPT Changes . . . 30
4.3.1 Learned Positional Embeddings . . . 30
4.3.2 GELU Instead of ReLU . . . 31
4.3.3 Pre-Norm Instead of Post-Norm . . . 31
4.3.4 Weight Tying . . . 31
4.3.5 Dropout . . . 32
4.4 The Language Modeling Objective . . . 32
4.5 Our BabyGPT Architecture . . . 33
4.5.1 Components . . . 33
4.5.2 Forward Pass . . . 34
4.6 Parameter Count . . . 35
4.7 Parameter Initialization . . . 36
4.8 Building the Model . . . 36
4.8.1 Step 1: Configuration . . . 37
4.8.2 Step 2: The Model . . . 37
4.8.3 Step 3: Verify . . . 41

5 Training **43**
5.1 Character-Level Tokenization . . . 43
5.1.1 Byte Pair Encoding (BPE) . . . 44
5.1.2 Label Smoothing . . . 46
5.2 Building the Dataset . . . 47
5.2.1 Creating the Data Pipeline . . . 48
5.3 The Optimizer: AdamW . . . 51
5.4 Learning Rate Schedule: Cosine Annealing with Warmup . . . 52
5.5 The Training Loop . . . 52
5.5.1 Mixed Precision Training . . . 53
5.5.2 Gradient Clipping . . . 53
5.5.3 Clearing Gradients . . . 54
5.5.4 Periodic Evaluation . . . 54
5.5.5 Checkpointing . . . 54
5.5.6 Running Training . . . 55
5.5.7 Hyperparameters Summary . . . 60

6 Generation and Sampling **61**
6.1 Autoregressive Generation . . . 61
6.2 Greedy Decoding . . . 62
6.3 Temperature Scaling . . . 62
6.4 Top-k Sampling . . . 63
6.5 Top-p (Nucleus) Sampling – Concept . . . 64
6.6 The Generate Function . . . 64
6.6.1 Creating the Generation Script . . . 65
6.7 Example Outputs . . . 67

6.8 What Would Make It Better 68
6.9 Project Summary 68

Chapter 1

Attention from First Principles

1.1 Project Setup

Before we begin, create the project and install PyTorch:

Shell — run in your terminal

```
mkdir learn-attn-impl && cd learn-attn-impl
uv init
uv add torch
```

This gives you a Python environment with PyTorch (and the documents assume a CUDA device present). All demo code in Chapters 1–3 can be run in a Python REPL (`uv run python`). Starting in Chapter 4, we will create the actual project source files.

Checkpoint: Run `uv run python -c "import torch; print(torch.cuda.is_available())"` — it should print `True`.

This document derives scaled dot-product attention from the ground up, builds multi-head attention on top of it, and introduces the causal mask needed for language modeling. By the end, every line of `CausalSelfAttention` in `babygpt/model.py` should make sense.

1.2 What Problem Does Attention Solve?

Consider a sequence of T token embeddings, each a vector in $\mathbb{R}^d$. (How tokens become vectors is covered in Chapter 3, Section 5. For now, just accept that each token is represented as a d-dimensional vector.) Every position in the sequence carries some meaning, but meaning in language is deeply contextual — the word "bank" means something different depending on whether "river" or "account" appears nearby. So each position needs a mechanism to gather information from other positions.

Before attention, the dominant approach was recurrent neural networks (RNNs). An RNN processes a sequence left to right, maintaining a hidden state that gets updated at each step. This has two problems:

1. **Sequential bottleneck.** You cannot compute the output at position t until you have computed positions 1 through $t-1$. This kills parallelism on GPUs.

2. **Information decay.** Information from position 1 must survive through every hidden-state update to reach position T. In practice, gradients vanish or explode, and the model struggles with long-range dependencies despite techniques like gating (LSTM, GRU).

Attention solves both problems with a single idea: **let each position compute a weighted average of all positions, where the weights are determined dynamically by the content at those positions.** There is no sequential dependency — every position can attend to every other position in parallel. And there is no distance decay — position 1 is just as accessible as position $T-1$, if the content says it should be.

The rest of this document is about making that idea precise.

1.3 Dot Products as Similarity

The mechanism that determines "how much should position i attend to position j?" is the dot product.

Recall from linear algebra: for two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$,

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos(\theta)$$

where θ is the angle between them. When $\mathbf{a}$ and $\mathbf{b}$ point in similar directions (small θ), $\cos(\theta)$ is close to 1 and the dot product is large. When they are orthogonal, the dot product is zero. When they point in opposite directions, it is negative.

In attention, we assign each position two roles:

- A **query** vector $\mathbf{q}$ — "what am I looking for?"
- A **key** vector $\mathbf{k}$ — "what do I contain?"

The dot product $\mathbf{q} \cdot \mathbf{k}$ measures how well the query at one position matches the key at another. High dot product means high relevance.

In practice, $\mathbf{q}$ and $\mathbf{k}$ are not stored directly — they are produced by multiplying the token embedding $\mathbf{x}$ by learned weight matrices: $\mathbf{q} = \mathbf{x}\, W^Q$ and $\mathbf{k} = \mathbf{x}\, W^K$. The network learns these projections during training.

Concrete example. Suppose we have 3 positions with query and key vectors in $\mathbb{R}^4$:

Demo — try in a REPL

```
import torch

Q = torch.tensor([
    [1.0,  0.0,  1.0,  0.0],  # query at position 0
    [0.0,  1.0,  0.0,  1.0],  # query at position 1
    [1.0,  1.0,  0.0,  0.0],  # query at position 2
])

K = torch.tensor([
    [1.0,  0.0,  1.0,  0.0],  # key at position 0
    [0.0,  1.0,  0.0,  1.0],  # key at position 1
    [1.0,  0.0,  0.0,  1.0],  # key at position 2
])
```

```
scores = torch.matmul(Q, K.transpose(-2, -1))
print(scores)
```

Output:

Expected output

```
tensor([[2., 0., 1.],
        [0., 2., 1.],
        [1., 1., 1.]])
```

Reading row 0: position 0's query is most similar to position 0's key (score 2), completely orthogonal to position 1's key (score 0), and moderately similar to position 2's key (score 1). This is exactly what you would verify by hand — $\mathbf{q}_0 = [1, 0, 1, 0]$ and $\mathbf{k}_0 = [1, 0, 1, 0]$ share all their nonzero components, while $\mathbf{k}_1 = [0, 1, 0, 1]$ has nonzero components exactly where $\mathbf{q}_0$ is zero.

Tip: Why the transpose? A single dot product $\mathbf{q}_i \cdot \mathbf{k}_j$ gives one similarity score. We need *all* T^2 pairwise scores at once. `K.transpose(-2, -1)` flips K from (T, d_k) to (d_k, T), turning each key into a column. Then `torch.matmul(Q, K^T)` computes row × column for every (i, j) pair — entry (i, j) of the result is exactly $\mathbf{q}_i \cdot \mathbf{k}_j$. The matrix multiply is just all T^2 dot products in one shot.

Dimensions. We write d_k for the dimension of each query and key vector (the same d from Section 2). If Q is (T, d_k) and K is (T, d_k), then $QK^\top$ is $(T, d_k) \times (d_k, T) = (T, T)$. Each entry (i, j) in this matrix is the raw attention score from position i to position j. We get a score for every pair of positions.

1.4 Why Scale by $\sqrt{d_k}$

Those raw scores have a problem: their magnitude depends on d_k, and this matters a lot when we feed them into softmax.

Here is the variance argument. Suppose each component of $\mathbf{q}$ and $\mathbf{k}$ is drawn independently from $\mathcal{N}(0, 1)$. Then:

- Each product $q_i k_i$ has mean $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$ (since they are independent and zero-mean).
- Each product $q_i k_i$ has variance $\mathrm{Var}(q_i k_i) = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1 \cdot 1 = 1$.
- The dot product $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$. Since they are independent, $\mathrm{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$.
- The standard deviation is $\sqrt{d_k}$.

So the dot product is a zero-mean random variable with standard deviation $\sqrt{d_k}$. When $d_k = 64$ (a typical value in practice), the standard deviation is 8. Dot products routinely land in the range $[-16, +16]$ or wider.

Why does this matter? Consider what softmax does with large inputs. If one score is 10 and the rest are near 0, then $\exp(10) \approx 22026$ dominates the denominator, and the output is nearly one-hot. In the backward pass, the gradients of softmax near a one-hot output are very close to zero — the model is "confident" and stops learning. This is the softmax saturation problem.

The fix is to divide by $\sqrt{d_k}$ before applying softmax, normalizing the scores back to unit variance:

Demo — try in a REPL

```
import math

d_k = Q.size(-1)
scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
```

There is a nice way to think about this: dividing by $\sqrt{d_k}$ is equivalent to setting the softmax **temperature** to $\sqrt{d_k}$. Higher temperature makes the distribution more uniform (softer attention); lower temperature makes it more peaked. By scaling, we ensure the temperature is calibrated regardless of dimension, so the model starts training with reasonable gradient flow.

1.5 Softmax Turns Scores into Weights

After scaling, we have a (T, T) matrix of real-valued scores. We need to convert each row into a probability distribution — non-negative values that sum to 1 — so we can take a weighted average. Softmax does this:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Properties:

- Every output is in $(0, 1)$ — strictly positive, never exactly zero.
- The outputs sum to 1 across the dimension we apply softmax over.
- It is monotonic: larger inputs get larger probabilities.
- It is differentiable everywhere, which is necessary for backpropagation.

In code:

Demo — try in a REPL

```
import torch.nn.functional as F

weights = F.softmax(scores, dim=-1)  # (T, T)
```

The `dim=-1` means we apply softmax across the last dimension (the key dimension), so each row independently becomes a probability distribution. Row i tells us: "how much should position i attend to each other position?"

Tip: Think of softmax as two steps: (1) pass every element through e^x, which makes all values strictly positive *and* amplifies larger values exponentially; (2) divide each result by the sum of all of them, so the vector sums to 1. Step 1 is the nonlinearity that separates winners from losers; step 2 is just linear normalization.

Connection to temperature. More generally, softmax with temperature τ is $\text{softmax}(\mathbf{z}/\tau)$. When τ is small, the distribution sharpens toward a one-hot on the maximum element. When τ is large, the distribution flattens toward uniform. The $\sqrt{d_k}$ scaling is exactly this: we implicitly set $\tau = \sqrt{d_k}$, preventing the distribution from being too sharp at initialization.

1.6 The Full Attention Formula

We now have all the ingredients. Each position also has a **value** vector $\mathbf{v}$ — "what I contribute to others if they attend to me." Like queries and keys, it is a learned projection of the token embedding: $\mathbf{v} = \mathbf{x}\,W^V$. The full scaled dot-product attention formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Dimension walkthrough, step by step:

Expression	Shape	What it represents
Q	(T, d_k)	Query vectors for each position
K	(T, d_k)	Key vectors for each position
V	(T, d_k)	Value vectors for each position
$QK^\top$	(T, T)	Raw attention scores for all position pairs
$QK^\top/\sqrt{d_k}$	(T, T)	Scaled scores (unit variance)
$\text{softmax}(QK^\top/\sqrt{d_k})$	(T, T)	Attention weights (each row sums to 1)
$\text{softmax}(\cdots)\,V$	(T, d_k)	Weighted combination of value vectors

The output has shape (T, d_k). Each row i of the output is a weighted average of the value vectors, where the weights come from how well position i's query matches each position's key.

Complete implementation:

This standalone function shows the algorithm. Our project uses PyTorch's fused `F.scaled_dot_product_att` instead.

Demo — try in a REPL

```
import math
import torch
import torch.nn.functional as F

def attention(Q, K, V, mask=None):
    """Scaled dot-product attention.

    Args:
        Q: (*, T, d_k) query vectors
        K: (*, S, d_k) key vectors
        V: (*, S, d_k) value vectors
        mask: broadcastable to (*, T, S), 0 where attention is forbidden

    Returns:
        (*, T, d_k) attended values
    """
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V)
```

A few things to note:

- The $\star$ in the shape annotations means this works with arbitrary leading batch dimensions — useful when processing multiple sequences at once.
- The `mask` argument is for causal masking (Section 8). Where the mask is 0, we set the score to $-\infty$ before softmax, which makes the weight exactly 0.
- The function is pure matrix multiplications and elementwise operations — extremely GPU-friendly.

Let us run it. We reuse Q and K from Section 3 and define a simple V where each position's value is a distinct 2D vector:

Demo — try in a REPL

```
V = torch.tensor([
    [1.0, 0.0],  # value at position 0
    [0.0, 1.0],  # value at position 1
    [0.5, 0.5],  # value at position 2
])

out = attention(Q, K, V)
print(out)
```

Expected output

```
tensor([[0.6601, 0.3399],
        [0.3399, 0.6601],
        [0.5000, 0.5000]])
```

Interpreting each row:

- **Position 0** — its query best matches its own key (Section 3), so it pulls mostly from $V_0 = [1, 0]$. The output $[0.66, 0.34]$ leans heavily toward V_0.
- **Position 1** — symmetric case. Its query best matches K_1, so it pulls mostly from $V_1 = [0, 1]$. The output $[0.34, 0.66]$ leans toward V_1.
- **Position 2** — its query matches all three keys roughly equally (scores were all 1 in Section 3), so attention is nearly uniform. The output $[0.50, 0.50]$ is close to the simple average $\frac{1}{3}(V_0 + V_1 + V_2)$.

The big picture. The attention output is not the final embedding — it is a *context-aware update*. In the full transformer (Chapter 2), this output is added back to the original embedding via a residual connection: $\mathbf{x} \leftarrow \mathbf{x} + \text{Attention}(\mathbf{x})$. So each position's embedding starts as "what this token means in isolation" and gets enriched with "what the surrounding context says is relevant." Stack multiple layers of this, and the embeddings progressively absorb more and more contextual information.

Tip: Since d_k equals the embedding dimension, the attention output has the same shape as the input — (T, d_k) — so the residual add works directly.

1.7 Multi-Head Attention

A single set of Q, K, V projections computes a single attention pattern: one (T, T) matrix of weights. But "relevance" is not one-dimensional. Position j might be relevant to position i for syntactic reasons (it is the verb's subject), for semantic reasons (it discusses the same topic), for positional reasons (it is the previous word), and so on.

Multi-head attention addresses this by running multiple attention computations in parallel, each with its own learned projections, and then combining the results.

The math. We now write d_{model} for the full embedding dimension (what we called d in Sections 2–6). Given input X of shape (T, d_{model}), and h heads:

For each head i (where $i = 1, \ldots, h$):

$$\text{head}_i = \text{Attention}(XW_i^Q,\ XW_i^K,\ XW_i^V)$$

where W_i^Q, W_i^K, W_i^V are each (d_{model}, d_k) projection matrices and $d_k = d_{\text{model}}/h$. Note that d_k now means the *per-head* dimension, smaller than the full embedding dimension we called d_k in Sections 3–6.

Then concatenate and project:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

where W^O is $(d_{\text{model}}, d_{\text{model}})$.

Each head operates in a d_k-dimensional subspace. Head 1 might learn to attend based on syntactic structure; head 2 might attend to nearby positions; head 3 might track coreference. The output projection W^O learns to combine these different views.

The following snippets show how multi-head attention works step by step inside `CausalSelfAttention` in `babygpt/model.py`, which we will create in Chapter 4.

1. **Project to Q, K, V.** Three separate weight matrices W^Q, W^K, W^V, each of shape $(d_{\text{model}}, d_{\text{model}})$, produce the queries, keys, and values:

   Conceptual — illustrating the idea

   ```
   q = self.W_Q(x)   # (B, T, d_model)
   k = self.W_K(x)   # (B, T, d_model)
   v = self.W_V(x)   # (B, T, d_model)
   ```

2. **Reshape into heads.** We reshape each (B, T, d_{model}) tensor into $(B, T, n_{\text{head}}, d_k)$ and then transpose to $(B, n_{\text{head}}, T, d_k)$:

   ```
   q = q.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
   k = k.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
   v = v.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
   ```

   Now the tensor has shape $(B, n_{\text{head}}, T, d_k)$. The n_{head} dimension acts as an additional batch dimension — the (T, d_k) attention computation runs independently for each head.

3. **Run attention.** Standard scaled dot-product attention over the last two dimensions. Each head computes its own (T, T) weight matrix and its own (T, d_k) output:

```
scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(head_dim)
weights = F.softmax(scores, dim=-1)
y = torch.matmul(weights, v)
```

This is the same formula from Section 6, now applied per head. Section 8 adds the causal mask that restricts each position to only attend to the past.

4. **Reassemble.** After attention, y has shape $(B, n_{\text{head}}, T, d_k)$. We transpose to $(B, T, n_{\text{head}}, d_k)$ and then merge the last two dimensions back into d_{model} (since $n_{\text{head}} \times d_k = d_{\text{model}}$):

```
y = y.transpose(1, 2).contiguous().view(B, T, d_model)
```

Tip: `.contiguous()` is a PyTorch bookkeeping call, not a mathematical operation. `.transpose()` does not rearrange data in memory — it only changes how indices map to the underlying storage. But `.view()` requires the data to be laid out contiguously (no gaps or out-of-order strides). `.contiguous()` copies the tensor into a fresh, contiguous block so that `.view()` can safely reinterpret the shape.

5. **Output projection.** A final linear layer mixes the concatenated head outputs:

```
y = self.c_proj(y)  # (B, T, d_model)
```

Parameter count. We have four weight matrices of shape $(d_{\text{model}}, d_{\text{model}})$: W^Q, W^K, W^V, and the output projection W^O. Total: $4 \cdot d_{\text{model}}^2$ parameters (ignoring biases). For our BabyGPT with $d_{\text{model}} = 384$, that is $4 \times 384^2 = 589{,}824$ parameters per attention layer.

You can see this exact implementation in `babygpt/model.py` in the `CausalSelfAttention` class. The three projections are `self.W_Q`, `self.W_K`, `self.W_V`, and the output projection is `self.c_proj`. The attention computation uses the same scaling, masking, softmax, and matrix multiply from Section 6.

1.8 Causal (Autoregressive) Masking

Everything so far describes **bidirectional** attention: every position can attend to every other position. This is fine for tasks where you have the full input (e.g., classification, encoding). But for language modeling — predicting the next token given the previous ones — position t must not see positions $t+1, t+2, \ldots, T$. That would be cheating.

The solution is a **causal mask**: a lower-triangular matrix of ones.

Causal mask pattern:

Expected output

```
1 0 0 0
1 1 0 0
1 1 1 0
1 1 1 1
```

Row t has ones in columns 0 through t and zeros elsewhere. Position 0 can only see itself. Position 1 can see positions 0 and 1. Position $T-1$ can see everything.

We apply this mask to the attention scores **before** softmax. Where the mask is 0 (future positions), we set the score to $-\infty$:

Demo — try in a REPL

```
mask = torch.tril(torch.ones(T, T))  # lower-triangular
scores = scores.masked_fill(mask == 0, float('-inf'))
```

Why $-\infty$ and not 0? Because of how softmax works. If we set a score to 0, then $\exp(0) = 1$, which is a perfectly valid nonzero weight — the model would still attend to the future position. We need the weight to be exactly zero, and the only way to get softmax to output zero is to feed it $-\infty$: $\exp(-\infty) = 0$. The masked position contributes nothing to the weighted average.

After masking and softmax, the weight matrix looks like:

Expected output

```
[1.00  0.00  0.00  0.00]
[0.45  0.55  0.00  0.00]
[0.20  0.30  0.50  0.00]
[0.10  0.25  0.35  0.30]
```

Each row sums to 1, and all entries above the diagonal are exactly 0. Every position attends only to itself and to the past.

In `babygpt/model.py`, the causal mask is constructed with `torch.tril` and applied before softmax, exactly as shown above:

This is how it appears in the final `babygpt/model.py`.

Conceptual — illustrating the idea

```
scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)
mask = torch.tril(torch.ones(T, T, device=x.device, dtype=torch.bool))
scores = scores.masked_fill(~mask, float('-inf'))
weights = F.softmax(scores, dim=-1)
weights = self.attn_dropout(weights)
y = torch.matmul(weights, v)
```

This is the complete attention computation from `CausalSelfAttention.forward` — scaling, masking, softmax, dropout, and the weighted sum of values, all visible.

1.9 Looking Ahead

We have now covered the core computational primitive of the transformer: scaled dot-product attention, extended to multiple heads, with causal masking for autoregressive modeling.

But attention alone is not a transformer. It is one sublayer in a larger block that also includes:

- **Residual connections** — the input to each sublayer is added to its output, enabling gradient flow through deep networks.
- **Layer normalization** — stabilizes training by normalizing activations.

- **A feed-forward network** — a position-wise MLP that gives each position capacity to process its attended representation.

The next chapter, *The Transformer Block*, shows how these components wrap around attention to form the repeating unit of the transformer architecture.

Chapter 2

The Transformer Block

Chapter 1 built up the core operation of a transformer: scaled dot-product attention, multi-head attention, and causal masking. But attention alone is not enough to build a deep network. This document covers the remaining components that, together with attention, form the **transformer block** — the repeating unit that gets stacked to build every modern large language model.

We will build four things: residual connections, layer normalization, the position-wise feed-forward network, and then assemble them into a single block.

2.1 Residual Connections

2.1.1 The degradation problem

A naive approach to building a more powerful network is to stack more layers. In theory, a deeper network can represent everything a shallower one can (the extra layers could just learn the identity function). In practice, He et al. (2015) showed that this does not happen: beyond a certain depth, adding layers actually *increases* training error. This is not overfitting — it is an optimization failure. The network is harder to train, and gradient-based optimization gets stuck.

The root cause is that gradients must pass through many multiplicative layers during backpropagation. Each layer multiplies the gradient by its Jacobian. If those Jacobians have eigenvalues consistently less than 1, gradients shrink exponentially (vanishing gradients). If greater than 1, they explode. Either way, the optimizer receives a poor signal about how to update early layers.

2.1.2 The ResNet fix

Instead of asking a sublayer to learn a target function $H(x)$, we ask it to learn the *residual* $F(x) = H(x) - x$:

$$\text{output} = x + F(x)$$

This is a residual connection (also called a skip connection). The sublayer only needs to learn what to *add* to the input, not how to reconstruct it from scratch.

Why does this help gradient flow? Take the gradient of the output with respect to the input:

$$\frac{\partial}{\partial x}\left[x + F(x)\right] = I + \frac{\partial F}{\partial x}$$

The identity matrix I guarantees that even if $\partial F/\partial x$ is very small, the gradient is at least I. In a deep stack of N blocks without residuals, the gradient must pass through N multiplicative factors — it can vanish or explode. With residuals, there is always a "shortcut" path where the gradient is 1. Information from the loss can reach the earliest layers without being distorted by intervening computations.

In code, a residual connection is one line:

Conceptual — illustrating the idea

```
x = x + sublayer(x)
```

Every sublayer in a transformer (attention and feed-forward) is wrapped with a residual connection.

It is useful to think of the tensor flowing through the network as a **residual stream**. Each sublayer reads from the stream, computes a delta, and writes (adds) that delta back. The stream accumulates contributions from every sublayer, and the residual connections ensure that no sublayer can destroy what previous sublayers have written — it can only add to it.

2.2 Layer Normalization

Deep networks train better when each layer's input has a stable distribution. Normalization layers address this by standardizing activations.

2.2.1 Why not batch normalization?

Batch normalization (Ioffe & Szegedy, 2015) computes the mean and variance of each feature across the batch dimension. It works well for CNNs, but it is a poor fit for sequence models:

1. **Variable-length sequences.** Different samples in a batch may have different lengths. Computing batch statistics over padded positions mixes real data with padding.
2. **Position-dependent distributions.** Position 1 of every sequence in a batch may have a very different distribution than position 50. Averaging across the batch conflates these.
3. **Inference with batch_size=1.** During generation we process one sequence at a time. Batch statistics over a single example are meaningless.

2.2.2 Layer normalization

Layer normalization (Ba et al., 2016) sidesteps all of these problems by normalizing across the **feature dimension** for each individual position, independently.

Given an input vector $\mathbf{x}$ of shape $(\ldots, d)$:

1. Compute the mean across features:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

2. Compute the variance across features:

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

3. Normalize, scale, and shift:

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

where:

- γ (weight) and β (bias) are learnable parameters of shape $(d,)$.
- ε is a small constant (typically 10^{-5}) to prevent division by zero.

The normalization centers and scales each position's feature vector to zero mean and unit variance. The learnable γ and β then let the model undo this if needed. Each position in the sequence is normalized independently, using only its own d-dimensional feature vector.

2.2.3 Implementation

Following modern convention (GPT-J, LLaMA), we default to `bias=False`:

This class will go into `babygpt/model.py` in Chapter 4. For now, understand the structure.

Conceptual — illustrating the idea

```
class LayerNorm(nn.Module):
    def __init__(self, config, eps=1e-5):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(config.n_embd))     # gamma
        self.bias = nn.Parameter(torch.zeros(config.n_embd)) if
           config.bias else None  # beta
        self.eps = eps

    def forward(self, x):
        # mean and variance over the last (feature) dimension
        mu = x.mean(dim=-1, keepdim=True)            # (B, T, 1)
        sigma_sq = x.var(dim=-1, keepdim=True,
                         unbiased=False)              # (B, T, 1)

        # normalize to zero mean, unit variance
        x_normalized = (x - mu) / torch.sqrt(sigma_sq + self.eps)

        # learnable scale and shift
        out = self.weight * x_normalized
        if self.bias is not None:
            out = out + self.bias
        return out
```

Tip: Why `unbiased=False`? The variance formula in the paper divides by d, not $d-1$. PyTorch's `Tensor.var` uses Bessel's correction $(d-1)$ by default. We pass `unbiased=False` to get the population variance that matches the LayerNorm definition. `F.layer_norm` does this internally — now you can see it explicitly.

2.3 Pre-Norm vs Post-Norm

Where you place the LayerNorm relative to the sublayer and the residual connection matters for training stability.

Post-norm (the original "Attention Is All You Need" paper):

$$x = \text{LayerNorm}(x + \text{sublayer}(x))$$

Pre-norm (GPT-2, GPT-3, and most modern transformers):

$$x = x + \text{sublayer}(\text{LayerNorm}(x))$$

2.3.1 Why pre-norm won

In post-norm, the normalization happens after the residual addition. During early training, the residual x and the sublayer output can have very different magnitudes. Their sum may have large variance, and the normalization must correct for this mismatch at every layer.

In pre-norm, the sublayer always receives a well-conditioned input (zero mean, unit variance) regardless of what earlier layers have done. This decouples the sublayer's optimization from the scale of the residual stream. Training is more stable, especially for deep models.

One consequence of pre-norm: the output of the last transformer block is a residual addition, but there is no normalization applied after that addition. So we need a **final LayerNorm** after the entire stack of blocks.

We use pre-norm throughout BabyGPT.

	Post-norm	Pre-norm
Formula	$x = \text{LN}(x + \text{sublayer}(x))$	$x = x + \text{sublayer}(\text{LN}(x))$
Used by	Original transformer	GPT-2, GPT-3, LLaMA
Final LN needed?	No	Yes
Training stability	Requires careful warmup	More stable out of the box

2.4 Position-wise Feed-Forward Network

Attention is the inter-position operation: it lets each position gather information from other positions. The feed-forward network (FFN) is the intra-position operation: it processes each position's representation independently, with a larger capacity.

2.4.1 Architecture

The FFN is a simple two-layer MLP applied to each position:

$$\text{FFN}(\mathbf{x}) = W_2 \cdot \text{GELU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

where W_1 has shape $(d_{\text{model}}, d_{\text{ff}})$ with $d_{\text{ff}} = 4 \cdot d_{\text{model}}$, and W_2 has shape $(d_{\text{ff}}, d_{\text{model}})$.

Tip: The 4x expansion. The FFN temporarily expands to $4 \times d_{\text{model}}$ dimensions. This gives each position a much wider intermediate representation — more capacity to extract and transform features. The expansion factor 4 is an empirical choice from the original paper that has become standard.

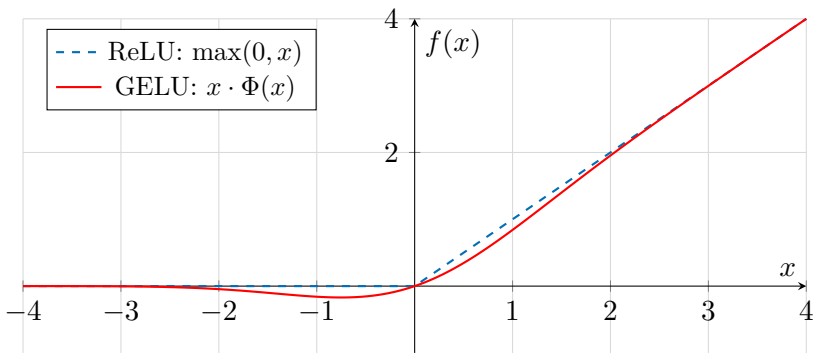
2.4.2 Activation function

The original transformer uses ReLU: $\max(0, x)$. GPT-2 and most modern language models use GELU:

$$\mathrm{GELU}(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the standard Gaussian CDF. Unlike ReLU, which has a hard cutoff at zero, GELU is smooth everywhere — negative inputs are dampened rather than zeroed out.

Tip: What is $\Phi(x)$? The standard Gaussian CDF is: $\Phi(x) = \frac{1}{2}\left[1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$ where erf is the error function. $\Phi(x)$ gives the probability that a standard normal random variable is $\leq x$. It rises smoothly from 0 to 1: $\Phi(-\infty) = 0$, $\Phi(0) = 0.5$, $\Phi(\infty) = 1$. Multiplying by x means large positive inputs pass through nearly unchanged ($\Phi \approx 1$), while large negative inputs are suppressed ($\Phi \approx 0$). In practice, PyTorch's `F.gelu` uses a fast tanh-based approximation: $\mathrm{GELU}(x) \approx 0.5\,x\left[1 + \tanh\left(\sqrt{2/\pi}\,(x + 0.044715\,x^3)\right)\right]$.



2.4.3 Why "position-wise"

The same weights W_1 and W_2 are shared across all positions in the sequence. Each position is processed independently — there is no information flow between positions inside the FFN. It is equivalent to a 1×1 convolution in CNN terminology.

Tip: Shared weights, independent application. If X has shape (T, d_{model}), then XW_1 is $(T, d_{\text{model}}) \times (d_{\text{model}}, d_{\text{ff}}) = (T, d_{\text{ff}})$. Each row of X (one position) is multiplied by the same W_1. In PyTorch, `nn.Linear` broadcasts the matmul across the T dimension automatically — no loop over positions, and no separate FFN per position. This is why it is called "position-wise": one FFN, applied to each position the same way.

2.4.4 Implementation

This class will go into `babygpt/model.py` in Chapter 4. For now, understand the structure.

Conceptual — illustrating the idea

```
class FeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd,
            bias=config.bias)
```

```
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd,
            bias=config.bias)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x):
        x = F.gelu(self.c_fc(x))
        return self.dropout(self.c_proj(x))
```

2.5 Assembling the Transformer Block

A single transformer block combines everything:

1. Attention sublayer with residual and pre-norm: $x = x + \text{CausalSelfAttention}(\text{LayerNorm}(x))$
2. FFN sublayer with residual and pre-norm: $x = x + \text{FeedForward}(\text{LayerNorm}(x))$

The two sublayers serve complementary roles:

- **Attention** handles inter-position communication.
- **FFN** handles intra-position computation.

2.5.1 Implementation

This class will go into `babygpt/model.py` in Chapter 4. For now, understand the structure.

Conceptual — illustrating the idea

```
class TransformerBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.ln_1 = LayerNorm(config)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config)
        self.ffn = FeedForward(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.ffn(self.ln_2(x))
        return x
```

2.6 Stacking Blocks

A transformer model is N identical blocks applied in sequence. In our BabyGPT, $N = 6$. Each block has its own parameters (not shared).

After the last block, a final LayerNorm is applied:

Preview of the forward pass in `BabyGPT`, which we build in Chapter 4.

Conceptual — illustrating the idea

```
x = tok_emb + pos_emb
for block in self.blocks:
    x = block(x)
x = self.final_ln(x)
```

2.6.1 Parameter count

With our config ($d_{\text{model}} = 384$, $d_{\text{ff}} = 1536$, $h = 6$, bias = False):

- Attention: $3 \times (384 \times 384) + 384 \times 384 = 589{,}824$
- FFN: $384 \times 1536 + 1536 \times 384 = 1{,}179{,}648$
- LayerNorm: $2 \times 384 = 768$
- Total per block: 1,770,240 ($\sim$1.77M parameters)
- 6 blocks: $\sim$10.6M parameters

The FFN accounts for about two-thirds of each block's parameters.

2.7 Looking Ahead

We now have the complete building block. In Chapter 3, we look at the full encoder-decoder architecture from the original paper. In Chapter 4, we simplify to the decoder-only GPT architecture we implement.

Chapter 3

The Full Transformer (Encoder-Decoder)

By now you understand the building blocks: scaled dot-product attention, multi-head attention, feed-forward networks, residual connections, and layer normalization. Chapters 1 and 2 showed how these components form a transformer block. This document steps back to the original architecture from "Attention Is All You Need" (Vaswani et al., 2017) and explains how those blocks are arranged into a full encoder-decoder model for machine translation.

We are not building this architecture ourselves — our BabyGPT will use only the decoder half. But understanding the full picture is necessary to appreciate what GPT keeps, what it drops, and why.

3.1 The Sequence-to-Sequence Problem

Machine translation takes a sentence in one language and produces the equivalent sentence in another. More broadly, many problems have this shape: an input sequence goes in, and a different-length output sequence comes out. Summarization (long document in, short summary out), speech-to-text (audio frames in, words out), and even question answering (context + question in, answer out) are all sequence-to-sequence tasks.

The **encoder-decoder** paradigm for handling this predates transformers. Sutskever et al. (2014) used two RNNs: an encoder RNN reads the input sequence token by token and produces a final hidden state (a single vector), then a decoder RNN generates the output sequence from that vector one token at a time.

The problem is immediate: compressing an arbitrarily long input sentence into a single fixed-size vector is a severe bottleneck. Information is inevitably lost, and the effect gets worse as the input sequence grows. A 50-word sentence and a 5-word sentence both get squeezed into the same-size vector.

Bahdanau et al. (2015) solved this by introducing **attention** into the encoder-decoder RNN. Instead of using only the encoder's final hidden state, the decoder at each time step computes a weighted sum over *all* of the encoder's hidden states. The decoder learns to focus on the relevant parts of the input as it generates each output token. This was the original attention mechanism.

The transformer takes this idea and makes it the entire architecture. There are no recurrent connections at all — just attention, feed-forward networks, and residual connections. The encoder uses self-attention to build rich representations of the input, the decoder uses self-attention to

build representations of the output so far, and cross-attention lets the decoder read from the encoder. The result is a model that is more parallelizable and more powerful than RNN-based approaches.

3.2 The Transformer Encoder

The encoder is a stack of $N = 6$ identical layers. Each encoder layer has two sublayers:

1. **Multi-head self-attention** (bidirectional — no causal mask)
2. **Position-wise feed-forward network**

Both sublayers are wrapped with a residual connection and followed by layer normalization (the original paper uses post-norm: `LayerNorm(x + Sublayer(x))`).

The structure of the full encoder:

```
Input Tokens -> Embedding + Positional Encoding
    -> [EncoderLayer x N]
    -> Encoder Output (B, T_src, d_model)
```

B is the batch size (number of sentences processed in parallel), T_{src} is the length of the source sequence (number of input tokens after padding), and d_{model} is the embedding dimension — the width of every hidden representation in the model (512 in the base transformer).

The critical difference from the decoder blocks in Chapter 2: **there is no causal mask**. Every position can attend to every other position. This is **bidirectional** attention — position 5 can look at position 10, and position 10 can look at position 5.

Why bidirectional? The encoder processes the entire input at once. When translating "The cat sat on the mat" from English to German, the encoder already has the complete input sentence. There is no reason to hide any positions from each other. The word "cat" should be able to use context from "mat" at the end of the sentence, and vice versa. Restricting the encoder to only look leftward (as a causal mask does) would throw away useful information for no benefit.

The only masking in the encoder is a **source mask** (also called a padding mask). When training on batches of sentences, shorter sentences are padded to the length of the longest sentence in the batch. We mask out these padding positions so they do not contribute to attention — they are not real tokens and should not influence the representation of actual tokens.

The output of the encoder is a tensor of shape `(B, T_src, d_model)`: a sequence of contextualized representations, one per input position. After passing through 6 layers of bidirectional self-attention, each position's representation encodes information from the entire input sequence. Position 3's output vector is not just about the token at position 3 — it is a representation of that token in the context of all other tokens.

3.3 The Transformer Decoder

The decoder is also a stack of $N = 6$ identical layers, but each decoder layer has **three** sublayers (not two):

1. **Masked multi-head self-attention** (with causal mask — same as our GPT blocks from Chapter 2)
2. **Multi-head cross-attention** (queries from the decoder, keys and values from the encoder output)

3. **Position-wise feed-forward network**

All three sublayers have residual connections and layer normalization.

The decoder generates output tokens one at a time during inference. When translating to German, it generates "Die", then "Katze", then "saß", and so on. The causal mask in the first sublayer ensures that when generating token at position t, the decoder can only attend to positions 0 through t (no peeking at future tokens it has not generated yet). During training, **teacher forcing** feeds the correct target sequence shifted by one position, but the causal mask still prevents the model from cheating by looking ahead.

The structure of the full decoder:

```
Target Tokens -> Embedding + Positional Encoding
    -> [DecoderLayer x N]
    -> Linear -> Softmax -> Output Probabilities
```

3.4 Cross-Attention: How the Decoder Reads the Encoder

Cross-attention is the mechanism that connects encoder and decoder. It is the reason the decoder can generate output that is conditioned on the input — without it, the decoder would just be a language model with no knowledge of the source sentence.

Cross-attention uses the exact same multi-head attention mechanism from Chapter 1, but the queries, keys, and values come from different places:

- **Queries (Q):** from the decoder's current state (the output of the masked self-attention sublayer)
- **Keys (K) and Values (V):** from the encoder's final output

Concretely, for decoder layer l:

$$\text{CrossAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where $Q = H_{\text{dec}}^{(l)} W^Q$, $K = H_{\text{enc}} W^K$, $V = H_{\text{enc}} W^V$.

$H_{\text{dec}}^{(l)}$ is the decoder's hidden state after masked self-attention in layer l (shape `(B, T_tgt, d_model)`), and H_{enc} is the encoder's final output (shape `(B, T_src, d_model)`).

What this means: each decoder position computes attention over all encoder positions. The attention weight matrix has shape `(B, h, T_tgt, T_src)` — each target position assigns a weight to each source position. The decoder "asks questions" via its queries, and the encoder "answers" through its keys and values.

Consider translating "The cat sat on the mat" to "Die Katze saß auf der Matte." When the decoder is generating "Katze" (German for "cat"), the query vector at that position will, through the attention mechanism, produce high attention weights on "cat" in the encoder output. The decoder is learning a soft alignment between source and target.

There is **no causal mask** in cross-attention — every decoder position can attend to every encoder position. The entire input is known (the encoder has already processed it), so there is nothing to hide. The only mask applied in cross-attention is the source padding mask, same as in the encoder's self-attention.

This cross-attention mechanism is conceptually the same as Bahdanau attention from the RNN seq2seq world: the decoder attends to the full encoder output when generating each token. But here it is multi-headed (8 heads in the original paper) and computed without any recurrence, making it far more parallelizable.

3.5 Embeddings

Token embeddings are a learned lookup table that maps each token ID (an integer) to a dense vector of size d_{model}:

Conceptual — illustrating the idea

```
embedding = nn.Embedding(vocab_size, d_model)
# Input: (B, T) tensor of integer token IDs
# Output: (B, T, d_model) tensor of embedding vectors
```

Each row of the embedding matrix corresponds to one token in the vocabulary. These vectors are initialized randomly and learned during training. After training, tokens with similar meanings will tend to have similar embedding vectors.

The original paper uses a **shared embedding matrix** for both the encoder and decoder. This makes sense when the source and target languages share a vocabulary (common with subword tokenization schemes like BPE, which construct a joint vocabulary across both languages).

BPE is covered in detail in Chapter 5, Section 1. Sharing means the model uses one set of embedding vectors for, say, the token "the" regardless of whether it appears on the source side or the target side.

Tip: The $\sqrt{d_{\text{model}}}$ scaling. The paper multiplies embedding vectors by $\sqrt{d_{\text{model}}}$ before adding positional encodings. This is easy to miss but has a concrete reason. `nn.Embedding` initializes its weight matrix from a standard normal distribution, which gives vectors with entries roughly distributed as $\mathcal{N}(0, 1)$. The L2 norm of such a d_{model}-dimensional vector is approximately $\sqrt{d_{\text{model}}}$, meaning each component has magnitude roughly 1. But in practice, many implementations (including the paper's) initialize with variance closer to $1/d_{\text{model}}$, which gives per-component magnitudes of roughly $1/\sqrt{d_{\text{model}}}$. Meanwhile, the sinusoidal positional encodings (described next) have components bounded in $[-1, 1]$, so their per-component magnitude is ~1. Without scaling, the positional encodings would dominate the token embeddings. Multiplying by $\sqrt{d_{\text{model}}}$ brings the embedding magnitudes up to the same scale as the positional encodings, so neither overwhelms the other.

3.6 Positional Encoding

Attention is **permutation-invariant**: if you shuffle the input tokens, the set of output vectors is the same set (just shuffled the same way). Formally, for any permutation π:

$$\text{Attention}(\{x_1, x_2, x_3\}) = \text{Attention}(\{x_{\pi(1)}, x_{\pi(2)}, x_{\pi(3)}\})$$

Without positional information, "the cat sat on the mat" and "mat the on sat cat the" would produce identical representations. Word order clearly matters, so we must inject position information.

The original paper uses **fixed sinusoidal positional encodings**:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

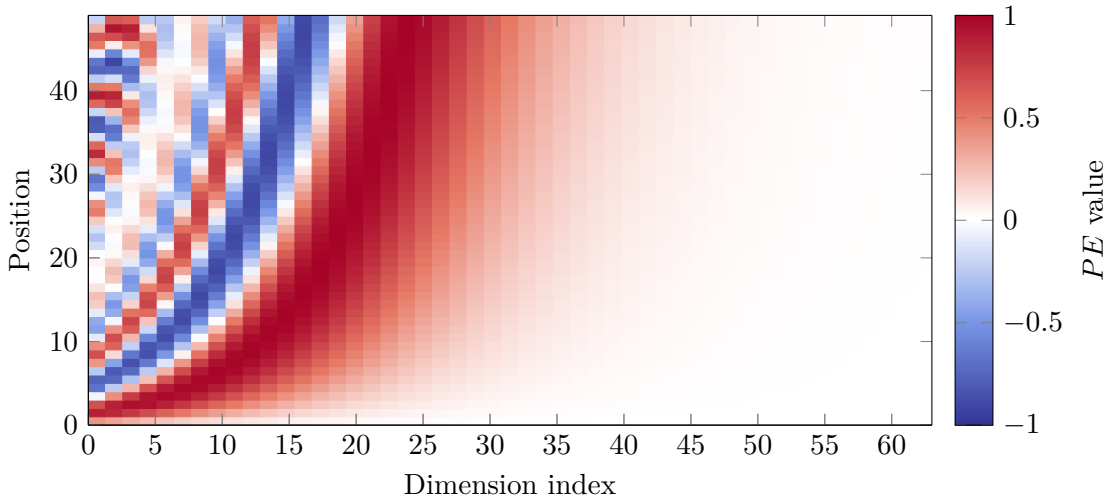
where pos is the position in the sequence (0, 1, 2, ...) and i is the dimension pair index (0, 1, 2, ..., $d_{\text{model}}/2 - 1$).

Tip: Reading the two formulas. The goal is to build a matrix PE of shape (T, d_{model}) — same shape as the token embeddings — so we can add them element-wise. The two formulas fill this matrix column by column: the first formula fills even columns $(0, 2, 4, \ldots)$ with sin, the second fills odd columns $(1, 3, 5, \ldots)$ with cos. Columns $2i$ and $2i{+}1$ form a pair that share the same frequency but use sin and cos respectively — like the x and y coordinates of a point on a circle. Each pair oscillates at a different frequency controlled by i: pair $i{=}0$ oscillates fast, and the last pair oscillates extremely slowly. The result is that each row (position) gets a unique combination of fast and slow oscillations across its d_{model} dimensions — a unique positional fingerprint. This is the matrix that gets added: `x = token_embeddings + PE`.

Each pair of dimensions $(2i, 2i+1)$ forms a sinusoid at a different frequency:

- Dimensions 0 and 1: wavelength $= 2\pi$ (the fastest oscillation — the encoding changes rapidly across positions)
- Last pair of dimensions: wavelength $= 2\pi \times 10000$ (extremely slow — the encoding barely changes from one position to the next)

The heatmap below shows the positional encoding matrix for 50 positions and 64 dimensions. Each row is one position, each column is one dimension, and color maps to value (-1 to $+1$). The left columns oscillate rapidly across positions (high frequency); the right columns change very slowly (low frequency). Each row has a unique pattern — the positional fingerprint.



The range of frequencies is geometric, spanning from 2π to $2\pi \times 10000$. This creates a unique "fingerprint" for each position: no two positions have the same pattern of values across all dimensions.

The geometric interpretation. Each dimension pair $(2i, 2i+1)$ can be viewed as a point on a circle rotating at frequency $1/10000^{2i/d_{\text{model}}}$. As position increases, the point traces the circle.

The key property is that $PE(pos + k)$ can be expressed as a linear transformation of $PE(pos)$ for any fixed offset k:

$$PE(pos + k) = M_k \cdot PE(pos)$$

where M_k is a rotation matrix that depends only on k, not on pos. This means the model can learn to compute relative positions using linear operations on these encodings.

Why sinusoidal and not learned? The paper argues that sinusoidal encodings may generalize to sequence lengths longer than those seen during training, since the sinusoidal functions are defined for any position. In practice, learned positional embeddings (as used in GPT-1 and GPT-2) work just as well for fixed-length contexts and are simpler to implement. Our BabyGPT in Chapter 4 will use learned positional embeddings.

A PyTorch implementation of sinusoidal positional encoding:

Demo — try in a REPL

This function is not part of our project — our GPT uses learned positional embeddings instead. Try it to visualize the encoding patterns.

```
import torch
import math

def sinusoidal_positional_encoding(max_len: int, d_model: int) ->
    torch.Tensor:
    """Compute sinusoidal positional encodings.

    Returns:
        (max_len, d_model) tensor of positional encodings.
    """
    pe = torch.zeros(max_len, d_model)
    position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        # (max_len, 1)
    # Compute the denominator in log-space for numerical stability:
    # 10000^(2i/d_model) = exp(2i/d_model * ln(10000))
    div_term = torch.exp(
        torch.arange(0, d_model, 2, dtype=torch.float) *
            (-math.log(10000.0) / d_model)
    )  # (d_model/2,)

    pe[:, 0::2] = torch.sin(position * div_term)  # even dimensions
    pe[:, 1::2] = torch.cos(position * div_term)  # odd dimensions
    return pe
```

The result is added element-wise to the token embeddings: `x = token_embedding * sqrt(d_model) + positional_encoding`. These encodings are not learned — they are computed once and remain fixed throughout training.

3.7 The Output Head (Generator)

After the stack of 6 decoder layers, we need to convert the decoder's output vectors (shape `(B, T_tgt, d_model)`) into a probability distribution over the vocabulary. This happens in two steps:

1. A **linear layer** projects from d_{model} to `vocab_size`: the output is a vector of raw scores (logits) for each token in the vocabulary.
2. **Softmax** converts these logits into a probability distribution.

Conceptual — illustrating the idea

```
# After decoder stack:
logits = linear(decoder_output)   # (B, T_tgt, vocab_size)
probs = softmax(logits, dim=-1)   # (B, T_tgt, vocab_size)
```

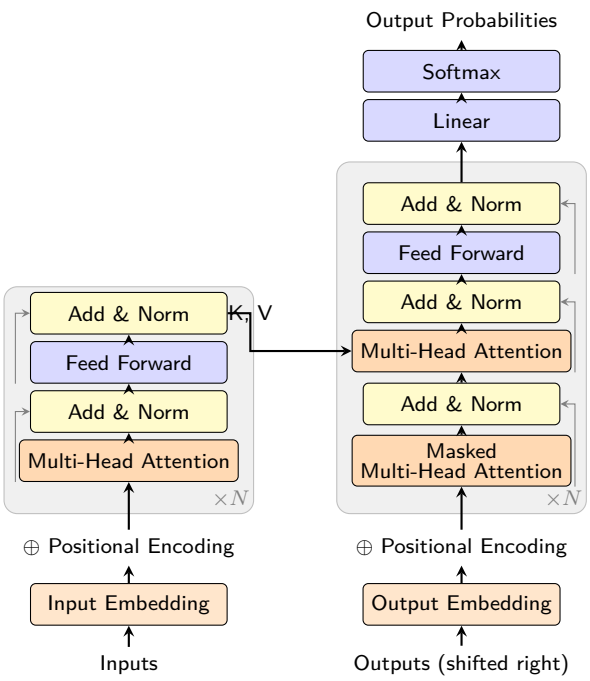
Weight tying. The original paper shares the weight matrix between the token embedding layer and this output projection layer. The embedding matrix has shape `(vocab_size, d_model)`. The output linear layer needs a weight matrix of shape `(d_model, vocab_size)` — which is exactly the transpose of the embedding matrix.

Why does this work? Consider what each does: the embedding layer maps a token ID to a vector in d_{model}-dimensional space. The output layer maps a d_{model}-dimensional vector to a score for each token in the vocabulary. When we use the transposed embedding matrix for the output projection, the score for a given token is the dot product between the decoder's output vector and that token's embedding vector. Tokens whose embeddings are similar to the decoder's output get high scores. This is a natural and consistent choice: the model's notion of "what a token means" (the embedding) is the same representation it uses to predict the next token.

Weight tying also has a practical benefit: for large vocabularies (32K or more tokens), the embedding matrix is a significant fraction of the model's total parameters. Sharing it with the output layer saves that entire parameter count.

3.8 Putting It All Together

The complete architecture, following the layout from "Attention Is All You Need" Figure 1:



Each `EncoderLayer` ($\times N$ on the left) contains:

- Multi-head self-attention (bidirectional) + residual + LayerNorm

- Feed-forward network + residual + LayerNorm

Each `DecoderLayer` ($\times N$ on the right) contains:

- Masked multi-head self-attention (causal) + residual + LayerNorm
- Multi-head cross-attention (Q from decoder, K/V from encoder) + residual + LayerNorm
- Feed-forward network + residual + LayerNorm

The original paper's hyperparameters:

Parameter	Base Model	Big Model
Layers (N)	6	6
d_{model}	512	1024
d_{ff} (FFN hidden dim)	2048	4096
Attention heads (h)	8	16
$d_k = d_v$ (per-head dim)	64	64
Total parameters	~65M	~213M

Training details:

- **Hardware:** 8 NVIDIA P100 GPUs
- **Training time:** ~3.5 days for the base model, ~12 hours per training step on 8 GPUs
- **Optimizer:** Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$) with a custom learning rate schedule:

$$lr = d_{\text{model}}^{-0.5} \cdot \min(step^{-0.5},\ step \cdot warmup_steps^{-1.5})$$

This schedule linearly increases the learning rate for the first `warmup_steps` (4000 steps), then decreases it proportionally to $1/\sqrt{step}$. The warmup prevents large, destabilizing updates early in training when the model's parameters are still random.

- **Regularization:** dropout of 0.1 applied to sublayer outputs, attention weights, and embeddings. Label smoothing of 0.1 (instead of training on hard 0/1 targets, the model trains on targets where the correct class gets probability 0.9 and the remaining 0.1 is spread across all other classes — this hurts perplexity slightly but improves BLEU score and generalization).

Results: On the WMT 2014 English-to-German translation benchmark, the base model achieved 27.3 BLEU and the big model achieved 28.4 BLEU — state of the art at the time, surpassing all previous models including ensembles.

3.9 Looking Ahead

The encoder-decoder architecture is well-suited for sequence-to-sequence tasks where the input and output are distinct sequences (translation, summarization). But for **language modeling** — the task of predicting the next token given preceding tokens — the encoder and cross-attention are unnecessary overhead. There is no separate "input" to encode; the model just reads text and predicts what comes next.

GPT (Radford et al., 2018) simplifies the transformer by keeping only the decoder (with its causal mask), removing the encoder and cross-attention entirely. What remains is a stack of transformer blocks, each containing masked self-attention and a feed-forward network — exactly the blocks we built in Chapter 2.

Chapter 4 makes this simplification explicit and assembles our BabyGPT.

Chapter 4

From the Transformer to GPT

The original transformer is a general-purpose sequence-to-sequence architecture. GPT is what happens when you take the decoder half, throw away everything else, and train it to predict the next token. This document traces that path: which pieces survive, what changes, and why those changes matter.

4.1 Three Transformer Variants

The "Attention Is All You Need" architecture spawned three distinct families, each keeping a different subset of the original design.

4.1.1 Encoder-Decoder (Vaswani et al., 2017)

This is the full architecture from Chapter 3. The encoder processes the source sequence with bidirectional self-attention (every token attends to every other token). The decoder generates the target sequence one token at a time with causal self-attention (each token attends only to earlier tokens), plus cross-attention layers that let the decoder read the encoder's output.

Use case: sequence-to-sequence tasks where the input and output are structurally different. Machine translation, summarization, speech-to-text.

4.1.2 Encoder-Only (BERT, Devlin et al., 2018)

Keep only the encoder stack. Every token attends to every other token in both directions. There is no autoregressive generation — the model processes the entire sequence at once.

Training objective: **masked language modeling (MLM)**. Randomly mask 15% of input tokens, and the model predicts what the masked tokens were. This forces the model to build rich bidirectional representations because every token needs context from both sides to predict the masked positions.

Use case: understanding tasks where you have the full input available — text classification, named entity recognition, extractive question answering. BERT and its descendants (RoBERTa, ALBERT, DeBERTa) dominated NLP benchmarks from 2018 to 2022.

4.1.3 Decoder-Only (GPT, Radford et al., 2018)

Keep only the decoder stack, and remove the cross-attention sublayer (since there is no encoder to attend to). Every layer is just causal self-attention followed by a feed-forward network — exactly the transformer block from Chapter 2.

Training objective: **next-token prediction**. Given the sequence so far, predict the next token. The causal mask ensures each position can only see tokens before it.

Use case: generation. But here is the key insight that made decoder-only models dominant: **any task can be framed as "continue this text."**

- Classification: "Review: 'This movie was terrible.' Sentiment:" → the model continues with "negative"
- Translation: "English: Hello. French:" → "Bonjour"
- Question answering: "Context: ... Question: Who won? Answer:" → "The home team"

GPT-2 demonstrated this with zero-shot task performance. GPT-3 (Brown et al., 2020) made it definitive with **in-context learning**: by placing a few examples in the prompt, the model could perform new tasks without any gradient updates. This "few-shot" capability scaled with model size and training data, and it is the reason decoder-only architectures became the foundation for modern large language models.

4.2 What GPT Removes

Recall from Chapter 3 that the original decoder layer has three sublayers:

1. Masked multi-head self-attention
2. Multi-head cross-attention (attending to encoder output)
3. Position-wise feed-forward network

GPT has no encoder, so there is nothing to cross-attend to. The cross-attention sublayer is deleted entirely. Each GPT layer has exactly two sublayers:

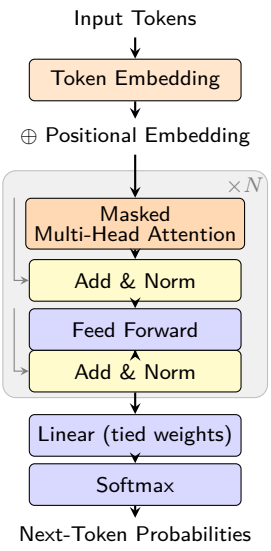
1. Masked multi-head self-attention
2. Position-wise feed-forward network

Here is the side-by-side comparison:

```
Original Decoder Layer (3 sublayers)     GPT Block (2 sublayers)
-------------------------------------    -------------------------
x                                        x
|                                        |
+-- Masked Self-Attention                +-- Masked Self-Attention
|   + Add & Norm                         |   + Add & Norm
|                                        |
+-- Cross-Attention (to encoder)         |  (removed -- no encoder)
|   + Add & Norm                         |
|                                        |
+-- Feed-Forward                         +-- Feed-Forward
|   + Add & Norm                         |   + Add & Norm
|                                        |
output                                   output
```

That is the entire structural difference. The self-attention and feed-forward sublayers work identically to what we built in Chapter 2.

Compare the decoder-only architecture to the full encoder-decoder from Chapter 3 — the entire left half and the cross-attention sublayer are gone:



4.3 What GPT Changes

Beyond removing cross-attention, GPT introduces several modifications that are now standard practice. None of these are architecturally dramatic, but they each solve a specific problem.

4.3.1 Learned Positional Embeddings

The original paper uses fixed sinusoidal positional encodings — the position signal is computed from a formula and added to the token embeddings. GPT replaces this with a learned positional embedding: a plain `nn.Embedding` table with one vector per position.

Conceptual — illustrating the idea

This line appears inside `BabyGPT.__init__` in `babygpt/model.py`.

```
# Sinusoidal (original paper): computed from a formula, not learned
# Learned (GPT): just another embedding table
self.wpe = nn.Embedding(context_size, n_embd)  # shape: (256, 384)
```

The learned embedding for position `t` is looked up and added to the token embedding, just like the sinusoidal encoding was. The difference is that the model learns the position representations from data rather than having them fixed by a formula.

Why this works: for a fixed maximum sequence length (our `context_size=256`), there is no advantage to the sinusoidal formula. A learned embedding has enough capacity to represent any position pattern the model needs. The sinusoidal encoding was designed to generalize to longer sequences than seen during training (via extrapolation of the sinusoidal functions), but in practice this extrapolation does not work well, and most models are trained and used within a fixed context window.

Tip: `nn.Embedding` vs. `nn.Linear`. An `Embedding(V, d)` is a lookup table: the input is an integer index, the output is the corresponding row of the weight matrix — no multiplication, just `weight[index]`. A `Linear(d_in, d_out)` is a matrix multiply: the input is a continuous vector, the output is `x @ weight.T + bias`. The connection: if you one-hot encoded a token index and multiplied by a weight matrix, you would select exactly one row — the same result as an Embedding lookup, but without materializing a sparse one-hot tensor. This is also why weight tying works: `lm_head` (a Linear) computes `hidden @ wte.weight.T`, which dot-products the hidden state against every embedding row — asking "which embedding vector is this hidden state closest to?"

4.3.2 GELU Instead of ReLU

The original transformer uses ReLU in the feed-forward network. GPT replaces it with GELU, which is smooth everywhere and does not hard-zero negative inputs. This was covered in detail in Chapter 2, Section 4. Our `FeedForward` module uses `F.gelu`.

4.3.3 Pre-Norm Instead of Post-Norm

The original paper applies LayerNorm *after* the residual addition (post-norm). GPT applies it *before* the sublayer (pre-norm), giving the residual connection a clean identity path for gradient flow. This was covered in detail in Chapter 2, Section 3. Our `TransformerBlock` uses pre-norm: `x = x + sublayer(layer_norm(x))`.

4.3.4 Weight Tying

The token embedding matrix `wte` has shape `(vocab_size, n_embd)` — it maps token IDs to embedding vectors. The language model head `lm_head` has shape `(n_embd, vocab_size)` — it maps the final hidden state back to logits over the vocabulary. These two matrices are transposes of each other conceptually: one maps from token space to embedding space, the other maps back.

Weight tying forces them to share the same weight matrix:

Conceptual — illustrating the idea

From `BabyGPT.__init__` in `babygpt/model.py`.

```
# In BabyGPT.__init__:
self.transformer.wte.weight = self.lm_head.weight
```

This means `lm_head` computes logits as a dot product between the hidden state and the token embeddings. A token's logit is high when the hidden state is close to that token's embedding vector. This reduces the parameter count (one less `vocab_size x n_embd` matrix) and acts as regularization — the model is encouraged to produce embeddings that are directly useful for prediction.

Tip: If `lm_head` shares weights with `wte`, how does it predict the *next* token? The hidden state at position t *starts* as the embedding of x_t, but 6 transformer blocks then transform it — attention mixes in context from positions $0 \ldots t$, and feed-forward layers apply nonlinear transformations. By the time it exits the final LayerNorm, it is no longer "the embedding of x_t." The loss function forces the transformer blocks to push that hidden vector **toward the embedding of** x_{t+1}, because the tied head computes $\text{logit}[i] = \text{dot}(\text{hidden}_t,\ \text{embedding}_i)$, and the loss rewards maximizing the logit for the correct next token. Think of the embedding space as a shared room where every character has a fixed address: the transformer blocks move the hidden state from near x_t's address toward x_{t+1}'s address, and the tied head asks "whose address are you closest to now?" Chapter 5 covers the loss function in detail.

4.3.5 Dropout

Dropout (Srivastava et al., 2014) is a regularization technique. During training, each activation is independently zeroed out with probability p:

$$\text{Dropout}(x)_i = \begin{cases} 0 & \text{with probability } p \\ x_i \,/\, (1-p) & \text{with probability } 1-p \end{cases}$$

The scaling by $1/(1-p)$ keeps the expected value of each element unchanged, so downstream layers see the same magnitude whether dropout is on or off.

Why this works: dropout prevents **co-adaptation** — neurons cannot rely on specific other neurons always being present, so the network learns more robust, redundant representations. Each forward pass trains a different random sub-network (some neurons removed, others kept), and they all share the same weights. The full network at inference time acts like an implicit ensemble of all those sub-networks.

During inference (`model.eval()`), dropout is disabled entirely — all activations pass through at full strength.

Our model uses $p = 0.2$ and applies dropout in three places:

1. **After the embedding addition** (`transformer.drop`): regularizes the combined token + position signal before it enters the transformer blocks.
2. **After attention weights** (`attn_dropout` in `CausalSelfAttention`): randomly drops connections between positions, forcing the model not to over-rely on any single attention pattern.
3. **After each sublayer's output projection** (`resid_dropout` in attention, `dropout` in FFN): regularizes the signal before it is added back via the residual connection.

The original transformer paper applies dropout in the same three locations with $p = 0.1$. We use a higher rate because our model is small and our dataset is small — stronger regularization helps prevent overfitting.

4.4 The Language Modeling Objective

GPT is trained with a single objective: predict the next token.

Given a sequence of tokens $x_1, x_2, \ldots, x_T$, the model learns to maximize the probability of each token given all preceding tokens. The loss function is the average negative log-likelihood:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

In plain English: for every position in the sequence, the model outputs a probability distribution over the vocabulary. The loss measures how much probability the model assigned to the token that actually came next, averaged over all positions. Lower loss means the model is better at predicting the next token.

This is computed as cross-entropy loss at every position simultaneously. The causal mask ensures that position t can only attend to positions $1, \ldots, t-1$, so each position's prediction genuinely uses only past context — even though all positions are processed in one forward pass.

This is **teacher forcing**: during training, each position sees the ground-truth tokens before it, not the model's own predictions. This allows the entire sequence to be processed in parallel (one forward pass), which is far more efficient than generating one token at a time.

Note how this differs from the encoder-decoder objective. In machine translation, you predict target tokens given a separate source sequence. In language modeling, the source and target are the same sequence, shifted by one position.

4.5 Our BabyGPT Architecture

The full model is in `babygpt/model.py`. Here is the architecture with our specific configuration from `babygpt/config.py`:

Parameter	Value	Meaning
`context_size`	256	Maximum sequence length (context window)
`n_layer`	6	Number of transformer blocks
`n_head`	6	Number of attention heads per block
`n_embd`	384	Embedding dimension (d_{model})
`dropout`	0.2	Dropout rate
`vocab_size`	65	Number of unique characters
`bias`	False	No bias terms in Linear/LayerNorm

4.5.1 Components

The model is organized as a `ModuleDict` called `transformer` plus a separate `lm_head`:

- **`wte`**: Token embedding — `nn.Embedding(65, 384)`. Maps each of the 65 character IDs to a 384-dimensional vector.
- **`wpe`**: Position embedding — `nn.Embedding(256, 384)`. Maps each position index (0 through 255) to a 384-dimensional vector.
- **`drop`**: Dropout applied after embedding addition.
- **`blocks`**: A `ModuleList` of 6 `TransformerBlock` layers. Each block contains a `CausalSelfAttention` and a `FeedForward` sublayer, each preceded by `LayerNorm` and followed by a residual connection.
- **`ln_f`**: A final `LayerNorm` applied after the last block (required for pre-norm architectures, since the residual path bypasses the per-block norms).
- **`lm_head`**: `nn.Linear(384, 65, bias=False)`. Projects the final hidden states to logits over the vocabulary. Weight-tied with `wte`.

Tip: Why `c_fc` and `c_proj`? These names come from OpenAI's GPT-2 codebase, which used 1D convolutions (`Conv1D`) instead of `nn.Linear` for these layers — a 1×1 convolution over the sequence dimension is mathematically identical to a per-position linear projection. The `c_` prefix stands for "convolution"; `fc` = fully connected (the expand layer), `proj` = projection (the contract/output layer). We keep the names so our weight dictionaries match GPT-2 checkpoints, which is useful if you ever want to load pretrained GPT-2 weights. The layers themselves are standard `nn.Linear`.

4.5.2 Forward Pass

Conceptual — illustrating the idea

From `BabyGPT.forward` in `babygpt/model.py`.

```
def forward(self, tokens: torch.Tensor, targets: torch.Tensor | None =
    None):
    batch, seq_len = tokens.shape
    positions = torch.arange(seq_len, device=tokens.device)

    token_emb = self.transformer.wte(tokens)        # (batch, seq_len,
        384)
    pos_emb = self.transformer.wpe(positions)        # (seq_len, 384) --
        broadcasts over batch
    x = self.transformer.drop(token_emb + pos_emb)

    for block in self.transformer.blocks:
        x = block(x)                                 # 6 transformer
            blocks

    x = self.transformer.ln_f(x)                     # final layer norm
    logits = self.lm_head(x)                         # (batch, seq_len,
        65)

    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.view(-1, logits.size(-1)),
            targets.view(-1),
        )
    return logits, loss
```

The input `tokens` has shape `(batch, seq_len)` — a batch of sequences, each `seq_len` tokens long. The output `logits` has shape `(batch, seq_len, 65)` — a probability distribution over 65 characters at every position. When `targets` are provided, cross-entropy loss is computed and returned alongside the logits.

Tip: Why does each position predict the *next* token? The causal mask ensures position t can only attend to tokens $x_0, \ldots, x_t$ — it cannot see x_{t+1} or beyond. So the hidden vector at position t is a learned summary of the past. The loss function (covered in Chapter 5) compares position t's output against the target x_{t+1}. Gradients flow backward and reshape the model so that each position's representation becomes useful for predicting what comes next — because that is the only thing the loss ever rewards. The architecture does not force this; training does.

4.6 Parameter Count

Let us count every trainable parameter in the model.

Token embedding (`wte`):

$$65 \times 384 = 24{,}960$$

Position embedding (`wpe`):

$$256 \times 384 = 98{,}304$$

Per transformer block:

The attention sublayer has three Q, K, V projections and one output projection:

- `W_Q`, `W_K`, `W_V`: $3 \times (384 \times 384) = 3 \times 147{,}456 = 442{,}368$
- `c_proj` (output): $384 \times 384 = 147{,}456$
- Total attention weights: $442{,}368 + 147{,}456 = 589{,}824$

Tip: How multi-head attention slices the matrices. Each `W_Q`, `W_K`, `W_V` projection is a single $(384, 384)$ matrix, but conceptually it is $h = 6$ separate $(384, 64)$ projections stacked side by side. The per-head dimension is $d_k = n_{\text{embd}}/h = 384/6 = 64$. In code, we project the full 384-dim vector first, then `view` the result as `(batch, seq_len, 6, 64)` and `transpose` to `(batch, 6, seq_len, 64)` — giving each head its own 64-dim Q, K, V to run attention on independently. After attention, the 6 heads of 64 dims are concatenated back to 384 and passed through `c_proj`. One big matrix multiply is equivalent to 6 small ones, but runs faster on a GPU because it is a single contiguous operation.

The feed-forward sublayer has two linear layers with a 4x expansion:

- `c_fc` (expand): $384 \times 1536 = 589{,}824$
- `c_proj` (contract): $1536 \times 384 = 589{,}824$
- Total FFN weights: $589{,}824 + 589{,}824 = 1{,}179{,}648$

Two LayerNorm layers (weight only, no bias since `bias=False`):

- $2 \times 384 = 768$

Total per block: $589{,}824 + 1{,}179{,}648 + 768 = 1{,}770{,}240$

All 6 blocks:

$$6 \times 1{,}770{,}240 = 10{,}621{,}440$$

Final LayerNorm (`ln_f`):

$$384$$

Language model head (`lm_head`): Weight-tied with `wte`, so 0 additional parameters.

Total:

$$24{,}960 + 98{,}304 + 10{,}621{,}440 + 384 = 10{,}745{,}088 \approx 10.7\text{M}$$

This is a small model by modern standards (GPT-3 is 175B parameters), but large enough to learn interesting character-level patterns from TinyShakespeare.

4.7 Parameter Initialization

Random initialization matters more than you might expect. If initial weights are too large, activations explode and training diverges. If too small, gradients vanish and the model learns nothing.

Standard weights: All weight matrices and embeddings are initialized from a normal distribution with mean 0 and standard deviation 0.02:

Conceptual — illustrating the idea
From `BabyGPT.__init__` and `BabyGPT._init_weights` in `babygpt/model.py`.

```
nn.init.normal_(module.weight, mean=0.0, std=0.02)
```

Biases: Initialized to zero (though our model has `bias=False`, so this only applies if you enable biases):

Conceptual — illustrating the idea
From `BabyGPT.__init__` and `BabyGPT._init_weights` in `babygpt/model.py`.

```
nn.init.zeros_(module.bias)
```

Residual projection scaling: The output projection (`c_proj`) in both the attention and feed-forward sublayers receives special treatment. These are the projections whose outputs get added back via the residual connection.

Conceptual — illustrating the idea
From `BabyGPT.__init__` in `babygpt/model.py`.

```
for param_name, param in self.named_parameters():
    if param_name.endswith("c_proj.weight"):
        nn.init.normal_(param, mean=0.0, std=0.02 / math.sqrt(2 *
            config.n_layer))
```

The scaling factor is $1/\sqrt{2N}$ where N is the number of transformer blocks.

Why this is necessary: consider what happens at initialization. Each transformer block adds the output of its two sublayers (attention and FFN) to the residual stream. If each sublayer's output has variance σ^2, then after N blocks with 2 residual additions each, the residual stream's variance has grown by approximately $2N \cdot \sigma^2$ (assuming the additions are roughly independent).

Scaling the residual branch outputs by $1/\sqrt{2N}$ reduces their variance by a factor of $2N$, so the total variance contribution from all residual additions is approximately $2N \cdot \sigma^2/(2N) = \sigma^2$. The residual stream's variance stays bounded regardless of depth.

For our model with $N = 6$: the scaling factor is $0.02/\sqrt{12} \approx 0.00577$. This is the GPT-2 initialization convention.

4.8 Building the Model

We now have everything we need to write the model code. Create two files:

4.8.1 Step 1: Configuration

File: babygpt/config.py

```
from dataclasses import dataclass


@dataclass
class GPTConfig:
    context_size: int = 256      # max sequence length (context window)
    vocab_size: int = 65         # number of unique tokens (characters in
        TinyShakespeare)
    n_layer: int = 6             # number of transformer blocks
    n_head: int = 6              # number of attention heads
    n_embd: int = 384            # embedding dimension (d_model)
    dropout: float = 0.2         # dropout rate (higher because dataset
        is small)
    bias: bool = False           # use bias in Linear layers and LayerNorm
```

4.8.2 Step 2: The Model

Create `babygpt/model.py` with the complete implementation. Every class from the previous chapters is collected here.

File: babygpt/model.py

```
"""BabyGPT: a decoder-only transformer language model."""

import math

import torch
import torch.nn as nn
import torch.nn.functional as F

from .config import GPTConfig


class LayerNorm(nn.Module):
    """Hand-written LayerNorm with optional bias.

    LayerNorm(x) = gamma * (x - mean) / sqrt(var + eps) + beta
    Normalizes across the last dimension (features) per position.
    gamma (weight) and beta (bias) are learnable affine parameters.
    """

    def __init__(self, config: GPTConfig, eps: float = 1e-5) -> None:
        super().__init__()
        self.weight = nn.Parameter(torch.ones(config.n_embd))     # gamma
        self.bias = nn.Parameter(torch.zeros(config.n_embd)) if
            config.bias else None  # beta
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        mean = x.mean(dim=-1, keepdim=True)
        variance = x.var(dim=-1, keepdim=True, unbiased=False)
        x_normalized = (x - mean) / torch.sqrt(variance + self.eps)
        out = self.weight * x_normalized
        if self.bias is not None:
```

```
            out = out + self.bias
        return out


class CausalSelfAttention(nn.Module):
    """Multi-head self-attention with causal mask.

    Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V
    Runs h heads in parallel by reshaping into (batch, h, seq_len, d_k).
    Causal mask sets future positions to -inf before softmax.
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        assert config.n_embd % config.n_head == 0
        # Each projection is (n_embd, n_embd), i.e. (384, 384).
        # Conceptually this is h separate (n_embd, head_dim) projections
        # stacked together: 6 heads x 64 dims = 384.
        self.W_Q = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.W_K = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.W_V = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.c_proj = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.attn_dropout = nn.Dropout(config.dropout)
        self.resid_dropout = nn.Dropout(config.dropout)
        self.n_head = config.n_head       # h = 6
        self.n_embd = config.n_embd       # 384
        self.head_dim = config.n_embd // config.n_head  # d_k = 384/6 =
            64

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        batch, seq_len, embed_dim = x.shape

        # Project then split into heads:
        # (batch, seq_len, 384) -> view as (batch, seq_len, 6, 64) ->
            (batch, 6, seq_len, 64)
        # Each head gets its own 64-dim slice of Q, K, V.
        query = self.W_Q(x).view(batch, seq_len, self.n_head,
            self.head_dim).transpose(1, 2)
        key = self.W_K(x).view(batch, seq_len, self.n_head,
            self.head_dim).transpose(1, 2)
        value = self.W_V(x).view(batch, seq_len, self.n_head,
            self.head_dim).transpose(1, 2)

        # Attention per head: (batch, 6, seq_len, 64) @ (batch, 6, 64,
            seq_len) -> (batch, 6, seq_len, seq_len)
        scores = torch.matmul(query, key.transpose(-2, -1)) /
            math.sqrt(self.head_dim)
        mask = torch.tril(torch.ones(seq_len, seq_len, device=x.device,
            dtype=torch.bool))
        scores = scores.masked_fill(~mask, float('-inf'))
        weights = F.softmax(scores, dim=-1)
        weights = self.attn_dropout(weights)
        attended = torch.matmul(weights, value)  # (batch, 6, seq_len,
            64)
```

```
        # Reassemble heads: (batch, 6, seq_len, 64) -> (batch, seq_len,
            6, 64) -> (batch, seq_len, 384)
        attended = attended.transpose(1, 2).contiguous().view(batch,
            seq_len, embed_dim)
        return self.resid_dropout(self.c_proj(attended))


class FeedForward(nn.Module):
    """Position-wise FFN with GELU.

    FFN(x) = W2 * GELU(W1 * x)
    Hidden dim is 4x the embedding dim (expand then contract).
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd,
            bias=config.bias)
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd,
            bias=config.bias)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.c_fc(x)
        x = F.gelu(x)
        x = self.c_proj(x)
        x = self.dropout(x)
        return x


class TransformerBlock(nn.Module):
    """Pre-norm transformer block.

    x = x + Attention(LayerNorm(x))    -- pre-norm + residual
    x = x + FFN(LayerNorm(x))          -- pre-norm + residual
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.ln_1 = LayerNorm(config)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config)
        self.ffn = FeedForward(config)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.attn(self.ln_1(x))
        x = x + self.ffn(self.ln_2(x))
        return x


class BabyGPT(nn.Module):
    """Decoder-only transformer language model.

    Forward: token_emb + pos_emb -> N x TransformerBlock -> LayerNorm ->
        lm_head
    Loss: cross-entropy over vocab at every position (next-token
        prediction).
```

```
    Weight tying: lm_head shares weights with token embedding.
    """

    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.config = config

        self.transformer = nn.ModuleDict(dict(
            wte=nn.Embedding(config.vocab_size, config.n_embd),
            wpe=nn.Embedding(config.context_size, config.n_embd),
            drop=nn.Dropout(config.dropout),
            blocks=nn.ModuleList(
                [TransformerBlock(config) for _ in range(config.n_layer)]
            ),
            ln_f=LayerNorm(config),
        ))
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size,
           bias=False)

        # Weight tying
        self.transformer.wte.weight = self.lm_head.weight

        # Initialize weights
        self.apply(self._init_weights)
        # Scaled init for residual projections (GPT-2 convention)
        for param_name, param in self.named_parameters():
            if param_name.endswith("c_proj.weight"):
                nn.init.normal_(
                    param, mean=0.0, std=0.02 / math.sqrt(2 *
                       config.n_layer)
                )

    def _init_weights(self, module: nn.Module) -> None:
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(
        self,
        tokens: torch.Tensor,
        targets: torch.Tensor | None = None,
    ) -> tuple[torch.Tensor, torch.Tensor | None]:
        batch, seq_len = tokens.shape
        assert seq_len <= self.config.context_size
        positions = torch.arange(seq_len, device=tokens.device)

        token_emb = self.transformer.wte(tokens)        # (batch,
           seq_len, n_embd)
        pos_emb = self.transformer.wpe(positions)        # (seq_len,
           n_embd)
        x = self.transformer.drop(token_emb + pos_emb)

        for block in self.transformer.blocks:
            x = block(x)
```

```python
        x = self.transformer.ln_f(x)
        logits = self.lm_head(x)                        # (batch,
            seq_len, vocab_size)

        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.view(-1, logits.size(-1)),
                targets.view(-1),
            )
        return logits, loss

    @torch.no_grad()
    def generate(
        self,
        tokens: torch.Tensor,
        max_new_tokens: int,
        temperature: float = 1.0,
        top_k: int | None = None,
    ) -> torch.Tensor:
        """Autoregressive token generation.

        Loop: forward pass -> logits / temperature -> top-k filter
              -> softmax -> multinomial sample -> append token.
        Crops to context_size when sequence exceeds context window.
        """
        for _ in range(max_new_tokens):
            tokens_cropped = (
                tokens if tokens.size(1) <= self.config.context_size
                else tokens[:, -self.config.context_size:]
            )

            logits, _ = self(tokens_cropped)
            logits = logits[:, -1, :] / temperature  # (batch,
                vocab_size)

            if top_k is not None:
                top_values, _ = torch.topk(logits, min(top_k,
                    logits.size(-1)))
                logits[logits < top_values[:, [-1]]] = float("-inf")

            probs = F.softmax(logits, dim=-1)
            next_token = torch.multinomial(probs, num_samples=1)  #
                (batch, 1)
            tokens = torch.cat([tokens, next_token], dim=1)

        return tokens

    def count_parameters(self) -> int:
        """Trainable parameters (excluding tied duplicates)."""
        total = sum(p.numel() for p in self.parameters())
        total -= self.lm_head.weight.numel()  # tied with wte
        return total
```

4.8.3 Step 3: Verify

Shell — run in your terminal

```
uv run python -c "
from babygpt.config import GPTConfig
from babygpt.model import BabyGPT
import torch
model = BabyGPT(GPTConfig())
print(f'Parameters: {model.count_parameters():,}')
x = torch.randint(0, 65, (2, 256))
logits, loss = model(x, torch.randint(0, 65, (2, 256)))
print(f'Logits: {logits.shape}, Loss: {loss.item():.2f}')
"
```

Checkpoint: You should see `Parameters: 10,720,128` and a loss near 4.17 (which is $\ln(65)$, the random baseline for 65-class classification).

Chapter 5

Training

This document covers everything that happens between having a model and having a trained model: the data pipeline, the optimizer, the learning rate schedule, and the training loop. Each component has a clear mathematical motivation.

5.1 Character-Level Tokenization

Before the model can process text, we need to convert characters to integers. TinyShakespeare contains approximately 1,115,394 characters drawn from a vocabulary of 65 unique characters (lowercase and uppercase letters, digits, punctuation, spaces, and newlines).

Our tokenizer (in `babygpt/tokenizer.py`) is the simplest possible: sort the unique characters, assign each an integer index, and store two lookup dictionaries.

File: `babygpt/tokenizer.py`

```python
"""Character-level tokenizer for TinyShakespeare."""


class CharTokenizer:
    """Maps individual characters to integers and back.

    This is the simplest possible tokenizer: the vocabulary is just the
        set of
    unique characters in the training text. Each character gets a unique
        integer
    ID. There are no subword units, no byte-pair encoding, no special
        tokens.
    """

    def __init__(self, text: str) -> None:
        chars = sorted(set(text))
        self.char_to_index = {ch: i for i, ch in enumerate(chars)}
        self.index_to_char = {i: ch for i, ch in enumerate(chars)}

    @property
    def vocab_size(self) -> int:
        return len(self.char_to_index)

    def encode(self, text: str) -> list[int]:
        return [self.char_to_index[ch] for ch in text]
```

```
    def decode(self, tokens: list[int]) -> str:
        return "".join(self.index_to_char[token] for token in tokens)
```

Encoding is a character-by-character lookup. Decoding is the reverse. That is the entire tokenizer.

5.1.1 Byte Pair Encoding (BPE)

Production language models do not tokenize character by character. They use **Byte Pair Encoding (BPE)** (Sennrich et al., 2016), a subword tokenization algorithm that finds a vocabulary of variable-length tokens by greedily merging frequent character pairs. The result sits between character-level and word-level tokenization: common words become single tokens, rare words are split into recognizable pieces, and any string can be represented.

The training algorithm

BPE learns its vocabulary from a training corpus. The algorithm is simple: start with a vocabulary of individual characters, then repeatedly merge the most frequent adjacent pair.

1. **Initialize.** Split every word in the corpus into a sequence of characters. The initial vocabulary is the set of all unique characters.
2. **Count pairs.** For every adjacent pair of tokens in every word, count how many times that pair appears across the entire corpus.
3. **Merge the top pair.** Find the pair with the highest count. Create a new token by concatenating the two tokens in the pair. Replace every occurrence of that pair in the corpus with the new token. Add the new token to the vocabulary.
4. **Repeat.** Go back to step 2. Stop after a predetermined number of merges (this controls the final vocabulary size).

Pseudocode:

Conceptual — illustrating the idea

```
def train_bpe(corpus: list[list[str]], num_merges: int) ->
    list[tuple[str, str]]:
    """Learn BPE merge rules from a tokenized corpus.

    corpus: list of words, each word is a list of character tokens.
            e.g. [["l","o","w"], ["l","o","w","e","r"], ...]
    Returns: ordered list of merge rules [(pair_a, pair_b), ...].
    """
    merges = []

    for i in range(num_merges):
        # Count every adjacent pair across all words
        pair_counts = {}
        for word in corpus:
            for j in range(len(word) - 1):
                pair = (word[j], word[j + 1])
                pair_counts[pair] = pair_counts.get(pair, 0) + 1

        if not pair_counts:
            break
```

```
        # Find the most frequent pair
        best_pair = max(pair_counts, key=pair_counts.get)
        merges.append(best_pair)

        # Merge: replace every occurrence of best_pair with a new token
        new_token = best_pair[0] + best_pair[1]
        for word in corpus:
            k = 0
            while k < len(word) - 1:
                if (word[k], word[k + 1]) == best_pair:
                    word[k:k + 2] = [new_token]
                else:
                    k += 1

    return merges
```

Worked example

Suppose the training corpus contains these words with their frequencies:

Word (as characters)	Frequency
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

Initial vocabulary: $\{\texttt{d}, \texttt{e}, \texttt{i}, \texttt{l}, \texttt{n}, \texttt{o}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}\}$ (10 tokens).

Merge 1. Count all adjacent pairs (weighted by word frequency):

$$\texttt{(e,s)} = 9 \quad \texttt{(s,t)} = 9 \quad \texttt{(l,o)} = 7 \quad \texttt{(o,w)} = 7 \quad \texttt{(n,e)} = 6 \quad \texttt{(e,w)} = 6 \quad \ldots$$

Tie between `(e,s)` and `(s,t)` at 9. Pick `(e,s)`. Merge into `es`. The corpus becomes:

$$\texttt{l o w}\ (5), \quad \texttt{l o w e r}\ (2), \quad \texttt{n e w es t}\ (6), \quad \texttt{w i d es t}\ (3)$$

Vocabulary: 11 tokens (added `es`).

Merge 2. Now `(es,t)` appears $6 + 3 = 9$ times — highest. Merge into `est`:

$$\texttt{l o w}\ (5), \quad \texttt{l o w e r}\ (2), \quad \texttt{n e w est}\ (6), \quad \texttt{w i d est}\ (3)$$

Merge 3. `(l,o)` appears 7 times. Merge into `lo`:

$$\texttt{lo w}\ (5), \quad \texttt{lo w e r}\ (2), \quad \texttt{n e w est}\ (6), \quad \texttt{w i d est}\ (3)$$

After more merges, common subwords like `low`, `new`, `est` become single tokens. Rare combinations stay as smaller pieces.

Encoding new text

Once the merge rules are learned, encoding a new string applies the same merges in order:

Conceptual — illustrating the idea

```
def encode_bpe(text: str, merges: list[tuple[str, str]]) -> list[str]:
    """Encode a string using learned BPE merge rules.

    Start with individual characters, then apply each merge rule
```

```
    in the order it was learned.
    """
    tokens = list(text)          # start as characters
    for pair in merges:
        new_token = pair[0] + pair[1]
        i = 0
        while i < len(tokens) - 1:
            if (tokens[i], tokens[i + 1]) == pair:
                tokens[i:i + 2] = [new_token]
            else:
                i += 1
    return tokens
```

For example, with the merges above, `"lowest"` encodes as `["low", "est"]` — two tokens instead of six characters.

Byte-level BPE (GPT-2)

GPT-2 uses a variant called **byte-level BPE**. Instead of starting with Unicode characters, the base vocabulary is all 256 individual bytes (`0x00` through `0xFF`). This guarantees that any input — including Unicode, emojis, or arbitrary binary data — can be tokenized without an "unknown token" fallback. GPT-2 performs approximately 50,000 merges, yielding a vocabulary of about 50,257 tokens.

Why we use character-level instead

Property	Character-level	BPE (subword)
Vocab size	65	30K – 50K
Sequence length	Long (1 char = 1 token)	Short (~4x fewer)
Tokenizer complexity	Two dictionaries	Merge table + algorithm
Info per token	1 character	~1 word or subword

BPE produces shorter sequences, which means the model's fixed context window covers more text. It also lets the model reason at word-level granularity rather than character-level. The trade-off is tokenizer complexity and a much larger embedding table.

We use character-level tokenization because it makes the full pipeline transparent. There are no merge rules, no special tokens, no tokenizer artifacts. Every step from raw text to model input is visible.

5.1.2 Label Smoothing

The original "Attention Is All You Need" paper trains with **label smoothing** ($\epsilon = 0.1$). Instead of a hard one-hot target, the correct class gets probability $1 - \epsilon + \epsilon/K$ and every other class gets ϵ/K:

$$\text{target}_i = \begin{cases} 1 - \epsilon + \epsilon/K & \text{if } i = \text{correct class} \\ \epsilon/K & \text{otherwise} \end{cases}$$

This prevents the model from pushing logits toward $\pm\infty$ to achieve a perfect one-hot output. Once the correct token reaches ~90% probability, there is no further gradient signal — the model is discouraged from over-fitting to individual training examples. We do not use label smoothing in BabyGPT; at this scale the benefit is minimal, and standard cross-entropy keeps the code simpler.

5.2 Building the Dataset

The data pipeline is in `babygpt/dataset.py`. The key steps:

These snippets show the logic inside `babygpt/dataset.py`, which you should create next.

1. Download and load. First, download the TinyShakespeare text file into the `data/` directory:

Shell — run in your terminal

```
mkdir -p data
URL=https://raw.githubusercontent.com/karpathy/char-rnn
URL=$URL/master/data/tinyshakespeare/input.txt
wget -O data/tinyshakespeare.txt $URL
```

Then read the file, encode all characters to integers, and store the result as a single long tensor.

Conceptual — illustrating the idea

```
text = DATA_FILE.read_text()                                # ~1.1M
    characters
tokenizer = CharTokenizer(text)
data = torch.tensor(tokenizer.encode(text), dtype=torch.long)  # shape:
    (1115394,)
```

2. Train/val split. The first 90% of the data is training, the last 10% is validation. We split sequentially (not randomly) because the data is a continuous text — random splitting would leak context across the boundary.

Conceptual — illustrating the idea

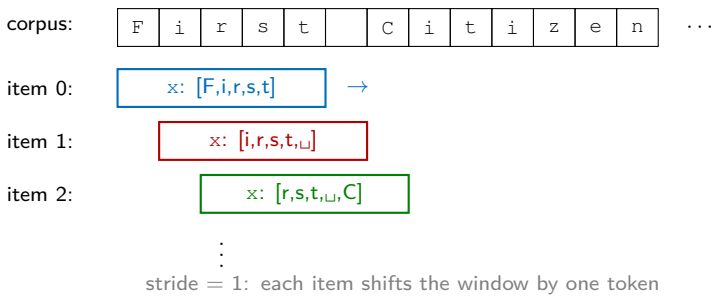
```
split = int(0.9 * len(data))
train_data = data[:split]    # ~1,003,854 tokens
val_data = data[split:]      # ~111,540 tokens
```

3. Create training examples. Each training example is a contiguous slice of `context_size + 1 = 257` tokens. The input `inputs` is the first 256 tokens; the target `targets` is the same window shifted right by one:

Conceptual — illustrating the idea

```
def __getitem__(self, index):
    chunk = self.data[index : index + self.context_size + 1]  # 257
        tokens
    inputs = chunk[:-1]    # tokens 0..255 -- the input
    targets = chunk[1:]      # tokens 1..256 -- the target
    return inputs, targets
```

The dataset generates examples by **striding with step size 1**: item 0 starts at position 0, item 1 starts at position 1, item 2 at position 2, and so on. Consecutive items overlap by 256 of 257 tokens. This means almost every token in the corpus appears as a target in many different contexts.



The total number of items is `len(data) - context_size`. For the training split: 1,003,854−256 = 1,003,598 overlapping windows. The `DataLoader` shuffles these and draws batches of 64.

Every position in `inputs` provides a training signal. Position 0 must predict position 1 (using only token 0 as context). Position 255 must predict position 256 (using all 256 tokens of context). A single `(inputs, targets)` pair gives us 256 next-token prediction examples.

4. DataLoader. Standard PyTorch DataLoader wraps the dataset:

Conceptual — illustrating the idea

```
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,           # random order for training
    pin_memory=True,        # faster CPU->GPU transfer
    drop_last=True,         # drop incomplete final batch
)
```

Each batch contains 64 sequences of length 256, giving $64 \times 256 = 16{,}384$ tokens per batch.

5.2.1 Creating the Data Pipeline

Create the tokenizer and dataset files. The tokenizer was shown above; here is the full dataset module:

File: babygpt/dataset.py

```
"""TinyShakespeare dataset: tokenize and create DataLoaders."""

from pathlib import Path

import torch
from torch.utils.data import Dataset, DataLoader

from .tokenizer import CharTokenizer
from .config import GPTConfig

DATA_FILE = Path(__file__).resolve().parent.parent / "data" /
    "tinyshakespeare.txt"


class ShakespeareDataset(Dataset):
    """Yields (input, target) pairs where input = chunk[:-1] and target
        = chunk[1:].

    Each sample is a contiguous slice of context_size+1 characters from
        the
```

```
    encoded text. The input is the first context_size tokens; the target
    is the same window shifted right by one position. Every position in
        the
    input provides one next-token prediction training example.
    """

    def __init__(self, data: torch.Tensor, context_size: int) -> None:
        self.data = data
        self.context_size = context_size

    def __len__(self) -> int:
        return len(self.data) - self.context_size

    def __getitem__(self, index: int) -> tuple[torch.Tensor,
        torch.Tensor]:
        chunk = self.data[index : index + self.context_size + 1]
        inputs = chunk[:-1]    # (context_size,)
        targets = chunk[1:]    # (context_size,)
        return inputs, targets


def get_dataloaders(
    config: GPTConfig,
    batch_size: int = 64,
    num_workers: int = 0,
) -> tuple[DataLoader, DataLoader, CharTokenizer]:
    """Read data, build tokenizer, return (train_loader, val_loader,
        tokenizer)."""
    text = DATA_FILE.read_text()
    tokenizer = CharTokenizer(text)

    # Encode entire text as a single long tensor
    data = torch.tensor(tokenizer.encode(text), dtype=torch.long)

    # 90/10 train/val split
    split = int(0.9 * len(data))
    train_data = data[:split]
    val_data = data[split:]

    train_dataset = ShakespeareDataset(train_data, config.context_size)
    val_dataset = ShakespeareDataset(val_data, config.context_size)

    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True,
        num_workers=num_workers,
        pin_memory=True,
        drop_last=True,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=num_workers,
        pin_memory=True,
        drop_last=True,
    )
```

```python
    return train_loader, val_loader, tokenizer


if __name__ == "__main__":
    text = DATA_FILE.read_text()
    tokenizer = CharTokenizer(text)
    print(f"Characters: {len(text):,}")
    print(f"Vocab size: {tokenizer.vocab_size}")
    print(f"Vocab: {''.join(sorted(tokenizer.char_to_index.keys()))!r}")

    data = torch.tensor(tokenizer.encode(text), dtype=torch.long)
    split = int(0.9 * len(data))
    print(f"Train tokens: {split:,}")
    print(f"Val tokens:   {len(data) - split:,}")
```

Shell — run in your terminal

```shell
uv run python -m babygpt.dataset
```

Checkpoint: You should see: `Characters: 1,115,394`, `Vocab size: 65`, `Train tokens: 1,003,854`, `Val tokens: 111,540`.

Inspecting a training sample. You can decode a sample back to text to see exactly what the model trains on:

Demo — try in a REPL

```python
from babygpt.config import GPTConfig
from babygpt.dataset import get_dataloaders

config = GPTConfig()
train_loader, val_loader, tok = get_dataloaders(config)

inputs, targets = next(iter(train_loader))   # one batch: (64, 256) each
print("Input: ", tok.decode(inputs[0, :40].tolist()))
print("Target:", tok.decode(targets[0, :40].tolist()))

# Output:
# Input:  First Citizen:\nBefore we proceed any
# Target: irst Citizen:\nBefore we proceed any f
```

The target is the input shifted by one character — at every position, the model learns to predict the next character.

Tip: Why does a shifted target teach next-token prediction — and how does weight tying fit in?
Two architectural constraints make this work. **(1) The causal mask** ensures position t only attends to tokens $x_0, \ldots, x_t$ — it cannot see x_{t+1} or beyond. So the hidden vector at position t is a learned summary of the past, not a copy of the future. **(2) The cross-entropy loss** compares position t's output against the target x_{t+1}. Gradients flow backward and reshape the model so that each position's hidden representation becomes useful for predicting what comes next — because that is the only thing the loss ever rewards.
Now recall that `lm_head` shares weights with `wte` (the token embedding matrix, shape 65×384). The logit for character i at position t is simply:

$$\text{logit}[t, i] = \text{dot}(\text{hidden}_t,\ \text{embedding}_i)$$

This is a similarity search: "which token's embedding is the hidden state closest to?" The hidden state at position t *starts* as the embedding of x_t (plus the positional embedding), but 6 transformer blocks then transform it — attention mixes in context from all positions $0 \ldots t$, and feed-forward layers apply nonlinear transformations. By the time the vector exits the final LayerNorm, it is no longer "the embedding of x_t."
The training loss forces the transformer blocks to push that hidden vector **toward the embedding of** x_{t+1}. Think of the embedding space as a shared room where every character has a fixed address: the transformer blocks move the hidden state from near x_t's address toward x_{t+1}'s address, and the tied output head asks "whose address are you closest to now?" Weight tying constrains input and output to share the same geometry — tokens that appear in similar contexts get similar embeddings *and* are predicted in similar contexts. This is a useful inductive bias, not a limitation.

5.3 The Optimizer: AdamW

AdamW (Loshchilov & Hutter, 2019) is the standard optimizer for transformer training. It combines two ideas:

1. **Adaptive learning rates.** Each parameter gets its own effective learning rate based on the history of its gradients. Parameters with consistently large gradients receive smaller updates; parameters with small gradients receive larger updates. This is the Adam part.
2. **Decoupled weight decay.** Weight matrices are shrunk toward zero by a fixed fraction each step ($\alpha\lambda$), independent of the gradient history. This regularization prevents any single weight from growing too large. The "decoupled" means the decay is applied separately from the adaptive gradient step, which is the key difference from naively adding L2 regularization to Adam.

The update rule:

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \alpha \cdot \lambda \cdot \theta_t$$

where $\hat{m}_t$ is the bias-corrected moving average of gradients (momentum), $\hat{v}_t$ is the bias-corrected moving average of squared gradients (adaptive scaling), and λ is the weight decay coefficient.

Not all parameters should be decayed. Weight matrices (2D tensors) get decay; biases and LayerNorm parameters (1D tensors) do not — they are offsets and scales that should be free to take whatever value is correct.

Parameter	Value
Learning rate (α)	1×10^{-3}
β_1 (momentum)	0.9
β_2 (adaptive scaling)	0.99
Weight decay (λ)	0.1 (2D params only)

5.4 Learning Rate Schedule: Cosine Annealing with Warmup

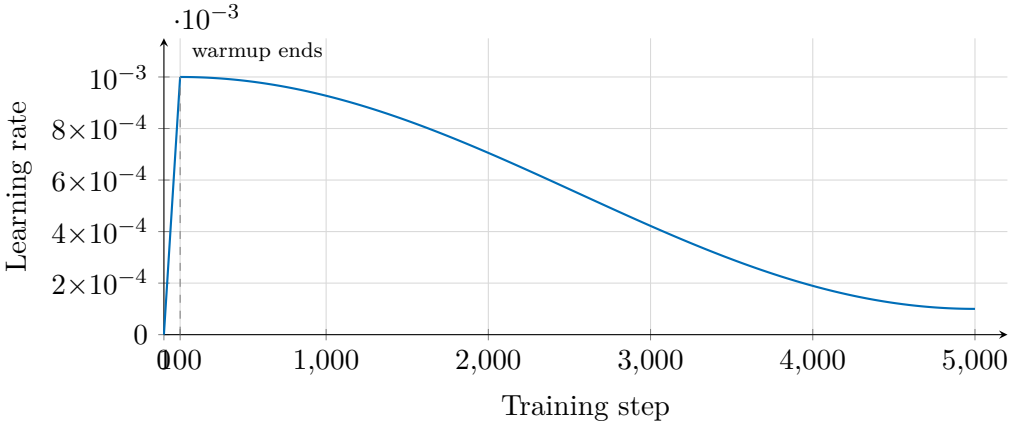
The learning rate follows a two-phase schedule: linear warmup, then cosine decay.

Warmup (steps 0–100): $\text{lr}(t) = \text{max_lr} \cdot t \,/\, \text{warmup_iters}$. The learning rate ramps linearly from 0 to the maximum. This gives AdamW's moment estimates time to stabilize before the optimizer takes large steps.

Cosine decay (steps 100–5000):

$$\text{lr}(t) = \text{min_lr} + \tfrac{1}{2}(\text{max_lr} - \text{min_lr})\big(1 + \cos(\pi \cdot \text{progress})\big)$$

where $\text{progress} = (t - \text{warmup_iters})/(\text{decay_iters} - \text{warmup_iters})$ goes from 0 to 1. The cosine curve spends more time near the maximum early on (when the model is making large progress) and near the minimum late in training (when fine adjustments matter).



5.5 The Training Loop

The training loop in `babygpt/train.py` follows the standard PyTorch pattern, with mixed precision and gradient clipping. Here is the core of the loop, annotated step by step:

Conceptual — illustrating the idea

From the main training loop in `babygpt/train.py`.

```
for step in range(max_iters):
    # 1. Set learning rate for this step
    lr = get_lr(step)
    for param_group in optimizer.param_groups:
        param_group["lr"] = lr

    # 2. Get a batch from the DataLoader, move to GPU
    inputs, targets = next(train_iter)    # inputs: (64, 256), targets:
        (64, 256)
    inputs, targets = inputs.to(device), targets.to(device)
```

```
    # 3. Forward pass with mixed precision
    with torch.amp.autocast("cuda", dtype=torch.float16,
        enabled=use_amp):
        _, loss = model(inputs, targets)

    # 4. Backward pass with loss scaling
    scaler.scale(loss).backward()

    # 5. Unscale gradients for correct norm computation
    scaler.unscale_(optimizer)

    # 6. Gradient clipping
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    # 7. Optimizer step and scaler update
    scaler.step(optimizer)
    scaler.update()

    # 8. Clear gradients
    optimizer.zero_grad(set_to_none=True)
```

Let us walk through each piece.

5.5.1 Mixed Precision Training

Modern GPUs have specialized hardware (tensor cores) that operate much faster on 16-bit floating point than on 32-bit. Mixed precision training exploits this:

- **Forward and backward pass:** Run in float16. This uses roughly half the memory and is faster on tensor cores.
- **Optimizer state and parameter updates:** Kept in float32. The optimizer needs full precision because it computes small corrections (learning rate times gradient divided by moment estimate) that would lose significant digits in float16.

The `torch.amp.autocast` context manager automatically casts operations to float16 where it is safe and leaves precision-sensitive operations (like softmax, loss computation) in float32.

The GradScaler: Float16 has a limited range — the smallest representable positive number is about 6×10^{-8}. Gradients for some parameters can be smaller than this, which means they round to zero (underflow). The GradScaler multiplies the loss by a large factor before `backward()`, which scales all gradients up into the representable range. Before the optimizer step, `scaler.unscale_()` divides the gradients back to their true values. The scale factor is adjusted dynamically: if gradients contain inf/nan (overflow), the scaler reduces the factor and skips the optimizer step.

5.5.2 Gradient Clipping

Conceptual — illustrating the idea

From the main training loop in `babygpt/train.py`.

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

This computes the global norm of all gradients:

$$\|g\| = \sqrt{\sum_i \|g_i\|^2}$$

where the sum is over all parameter tensors. If $\|g\| > \text{max_norm}$, every gradient is scaled down by $\text{max_norm}/\|g\|$ so that the global norm becomes exactly `max_norm`.

Why this matters: occasionally, a batch produces anomalously large gradients (an "exploding gradient"). Without clipping, a single bad batch can make a destructive parameter update that ruins training progress. Gradient clipping bounds the worst-case step size while leaving normal-sized gradients unchanged.

Note: `scaler.unscale_()` must be called before gradient clipping. Otherwise you would be clipping the scaled gradients, and the effective max norm would vary with the current scale factor.

5.5.3 Clearing Gradients

Conceptual — illustrating the idea

From the main training loop in `babygpt/train.py`.

```
optimizer.zero_grad(set_to_none=True)
```

PyTorch accumulates gradients by default — calling `.backward()` adds to the existing `.grad` attribute. We need to clear gradients before each step. The `set_to_none=True` argument sets `.grad = None` instead of filling with zeros. This is slightly faster (avoids a memset) and uses no memory until the next backward pass populates the gradients.

5.5.4 Periodic Evaluation

Every 250 steps, the training loop evaluates the model on both training and validation data by averaging the loss over 200 batches:

Conceptual — illustrating the idea

From the main training loop in `babygpt/train.py`.

```
@torch.no_grad()
def estimate_loss(model, train_loader, val_loader, device,
    eval_iters=200):
    model.eval()
    # ... average loss over eval_iters batches for each split ...
    model.train()
    return {"train": train_loss, "val": val_loss}
```

`model.eval()` disables dropout (all tokens are kept, but activations are not scaled — PyTorch handles this automatically). `@torch.no_grad()` disables gradient tracking, saving memory during evaluation. After evaluation, `model.train()` re-enables dropout.

5.5.5 Checkpointing

When the validation loss improves, the model state is saved:

Conceptual — illustrating the idea

From the main training loop in `babygpt/train.py`.

```
checkpoint = {
    "model": model.state_dict(),
    "config": config,
    "step": step,
    "val_loss": best_val_loss,
    "vocab": tokenizer.char_to_index,
}
torch.save(checkpoint, CHECKPOINT_DIR / "best.pt")
```

The checkpoint includes everything needed to reconstruct the model: the weights, the architecture config, and the vocabulary mapping. This is what `babygpt/generate.py` loads to produce text.

5.5.6 Running Training

Create `babygpt/train.py` with the complete training script:

File: `babygpt/train.py`

```
"""Training script for BabyGPT on TinyShakespeare."""

import argparse
import math
import time
from pathlib import Path

import torch

from .config import GPTConfig
from .dataset import get_dataloaders
from .model import BabyGPT

# ---- Hyperparameters ----
BATCH_SIZE = 64
MAX_ITERS = 5000
EVAL_INTERVAL = 250
EVAL_ITERS = 200

LEARNING_RATE = 1e-3
MIN_LR = 1e-4
WARMUP_ITERS = 100
LR_DECAY_ITERS = 5000

WEIGHT_DECAY = 0.1
BETA1 = 0.9
BETA2 = 0.99
GRAD_CLIP = 1.0

CHECKPOINT_DIR = Path(__file__).resolve().parent.parent / "checkpoints"


def get_lr(step: int) -> float:
    """Cosine annealing with linear warmup.

    - Warmup: linear ramp from 0 to LEARNING_RATE over WARMUP_ITERS
      steps.
    - Cosine decay: smoothly decrease from LEARNING_RATE to MIN_LR.
```

```
    Math: lr = min_lr + 0.5 * (max_lr - min_lr) * (1 + cos(pi *
        progress))
    where progress goes from 0 to 1 over the decay phase.
    """
    # Linear warmup
    if step < WARMUP_ITERS:
        return LEARNING_RATE * step / WARMUP_ITERS
    # After decay period, stay at min
    if step > LR_DECAY_ITERS:
        return MIN_LR
    # Cosine decay
    progress = (step - WARMUP_ITERS) / (LR_DECAY_ITERS - WARMUP_ITERS)
    coeff = 0.5 * (1.0 + math.cos(math.pi * progress))
    return MIN_LR + coeff * (LEARNING_RATE - MIN_LR)


def configure_optimizer(model: BabyGPT) -> torch.optim.AdamW:
    """Set up AdamW with separate parameter groups for weight decay.

    Weight matrices get decayed; biases, layer norm params, and
        embeddings don't.
    This is standard practice -- weight decay on biases/norms doesn't
        help and
    can hurt.
    """
    decay_params = []
    no_decay_params = []

    for name, param in model.named_parameters():
        if not param.requires_grad:
            continue
        # Don't decay 1D parameters (biases, layer norm weights)
        if param.dim() < 2:
            no_decay_params.append(param)
        else:
            decay_params.append(param)

    optim_groups = [
        {"params": decay_params, "weight_decay": WEIGHT_DECAY},
        {"params": no_decay_params, "weight_decay": 0.0},
    ]

    return torch.optim.AdamW(optim_groups, lr=LEARNING_RATE,
        betas=(BETA1, BETA2))


@torch.no_grad()
def estimate_loss(
    model: BabyGPT,
    train_loader: torch.utils.data.DataLoader,
    val_loader: torch.utils.data.DataLoader,
    device: torch.device,
    eval_iters: int = EVAL_ITERS,
) -> dict[str, float]:
    """Estimate train and val loss by averaging over eval_iters
        batches."""
    model.eval()
```

```
    results = {}
    for split_name, loader in [("train", train_loader), ("val",
        val_loader)]:
        losses = []
        loader_iter = iter(loader)
        for _ in range(eval_iters):
            try:
                inputs, targets = next(loader_iter)
            except StopIteration:
                loader_iter = iter(loader)
                inputs, targets = next(loader_iter)
            inputs, targets = inputs.to(device), targets.to(device)
            _, loss = model(inputs, targets)
            losses.append(loss.item())
        results[split_name] = sum(losses) / len(losses)
    model.train()
    return results


def train(args: argparse.Namespace) -> None:
    # Device
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")
    if device.type == "cuda":
        print(f"GPU: {torch.cuda.get_device_name()}")

    # Seed
    torch.manual_seed(42)
    if device.type == "cuda":
        torch.cuda.manual_seed(42)

    # Data
    config = GPTConfig()
    train_loader, val_loader, tokenizer = get_dataloaders(
        config, batch_size=BATCH_SIZE
    )
    # Update vocab_size from actual data
    config.vocab_size = tokenizer.vocab_size
    print(f"Vocab size: {config.vocab_size}")

    # Model
    model = BabyGPT(config).to(device)
    print(f"Model parameters: {model.count_parameters():,}")

    # Optimizer
    optimizer = configure_optimizer(model)

    # Mixed precision
    use_amp = device.type == "cuda"
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
    amp_dtype = torch.float16  # RTX 3080 supports bf16 too, but fp16 is
        safe default

    # Training loop
    CHECKPOINT_DIR.mkdir(parents=True, exist_ok=True)
    best_val_loss = float("inf")

    train_iter = iter(train_loader)
```

```
    start_time = time.time()

    for step in range(args.max_iters):
        # Set learning rate for this step
        lr = get_lr(step)
        for param_group in optimizer.param_groups:
            param_group["lr"] = lr

        # Get batch (cycle through loader)
        try:
            inputs, targets = next(train_iter)
        except StopIteration:
            train_iter = iter(train_loader)
            inputs, targets = next(train_iter)
        inputs, targets = inputs.to(device), targets.to(device)

        # Forward + backward with mixed precision
        with torch.amp.autocast("cuda", dtype=amp_dtype,
           enabled=use_amp):
            _, loss = model(inputs, targets)

        scaler.scale(loss).backward()

        # Gradient clipping (unscale first for correct norm computation)
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP)

        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad(set_to_none=True)

        # Logging and evaluation
        if step % EVAL_INTERVAL == 0 or step == args.max_iters - 1:
            losses = estimate_loss(model, train_loader, val_loader,
               device)
            elapsed = time.time() - start_time
            tokens_per_sec = (step + 1) * BATCH_SIZE *
               config.context_size / elapsed
            print(
                f"step {step:5d} | "
                f"train loss {losses['train']:.4f} | "
                f"val loss {losses['val']:.4f} | "
                f"lr {lr:.2e} | "
                f"tok/s {tokens_per_sec:.0f}"
            )

            # Save best checkpoint
            if losses["val"] < best_val_loss:
                best_val_loss = losses["val"]
                checkpoint = {
                    "model": model.state_dict(),
                    "config": config,
                    "step": step,
                    "val_loss": best_val_loss,
                    "vocab": tokenizer.char_to_index,
                }
                torch.save(checkpoint, CHECKPOINT_DIR / "best.pt")
                print(f"  -> saved checkpoint
```

```
                    (val_loss={best_val_loss:.4f})")

    # Save final checkpoint
    checkpoint = {
        "model": model.state_dict(),
        "config": config,
        "step": args.max_iters,
        "val_loss": best_val_loss,
        "vocab": tokenizer.char_to_index,
    }
    torch.save(checkpoint, CHECKPOINT_DIR / "final.pt")

    total_time = time.time() - start_time
    print(f"\nTraining complete in {total_time:.1f}s")
    print(f"Best val loss: {best_val_loss:.4f}")
    print(f"Checkpoints saved to {CHECKPOINT_DIR}")


def main() -> None:
    parser = argparse.ArgumentParser(description="Train BabyGPT on
        TinyShakespeare")
    parser.add_argument("--max-iters", type=int, default=MAX_ITERS)
    args = parser.parse_args()
    train(args)


if __name__ == "__main__":
    main()
```

Shell — run in your terminal

```
uv run python -m babygpt.train
```

Checkpoint: On an RTX 3080 (10 GB VRAM), training takes approximately **13 minutes** for 5000 iterations at ~**103K tokens/sec**, using ~**3.3 GB peak VRAM** with mixed precision. You should see the loss drop from ∼4.17 to a best validation loss around **1.3–1.5**. The best checkpoint is saved to `checkpoints/best.pt`.
Note: our implementation hand-rolls LayerNorm and attention using plain PyTorch operations rather than calling optimized fused kernels (e.g. `F.scaled_dot_product_attention`, `F.layer_norm`). This is intentional — seeing the raw matrix operations is the point of this project — but it means training is slower than an optimized implementation would be. A production codebase using fused kernels and FlashAttention would train the same model significantly faster.

5.5.7 Hyperparameters Summary

Parameter	Value	Rationale
`batch_size`	64	$64 \times 256 = 16{,}384$ tokens/batch. Fits in 10 GB VRAM.
`max_iters`	5000	≈82M tokens seen. Multiple passes over the dataset.
`learning_rate`	10^{-3}	Relatively high — small models tolerate larger LRs.
`min_lr`	10^{-4}	10x below max. Fine-tunes in the late phase.
`warmup_iters`	100	2% of training. Moment estimates stabilize.
`weight_decay`	0.1	Standard for transformers. Prevents weight growth.
`grad_clip`	1.0	Rarely triggers but prevents catastrophic steps.
`beta1`	0.9	Standard momentum coefficient.
`beta2`	0.99	Faster adaptation than default 0.999.

Next: Chapter 6 covers autoregressive generation, temperature, top-k sampling, and what to expect from our trained model.

Chapter 6

Generation and Sampling

Training teaches the model to predict the next token. Generation is where we actually use that ability: start with a prompt, predict the next token, append it, and repeat. The quality of the output depends entirely on how we select each token from the model's predicted probability distribution.

6.1 Autoregressive Generation

The generation procedure is a loop:

1. Start with a prompt (a sequence of tokens).
2. Feed the sequence to the model. The model outputs logits (unnormalized scores) for every position, but we only care about the logits at the last position — those are the predictions for the next token.
3. Convert logits to a probability distribution (via softmax, possibly with modifications).
4. Sample a token from that distribution.
5. Append the sampled token to the sequence.
6. Repeat from step 2.

This is inherently sequential. Each new token depends on all previous tokens (through the causal self-attention mechanism), so there is no way to generate multiple tokens in parallel. This is the fundamental speed bottleneck of autoregressive language models — during generation, you do one full forward pass per token.

When the sequence grows beyond `context_size` (256 in our model), we crop from the left:

Conceptual — illustrating the idea
From `BabyGPT.generate` in `babygpt/model.py`.

```
tokens_cropped = (
    tokens if tokens.size(1) <= self.config.context_size
    else tokens[:, -self.config.context_size:]
)
```

The model can only attend to the most recent 256 tokens. Earlier tokens are simply dropped. This is a hard limitation of fixed-length positional embeddings.

Two settings matter during generation:

- **`model.eval()`** disables dropout. During training, dropout randomly zeroes activations to prevent overfitting. During generation, we want deterministic, full-capacity behavior from the model.
- **`torch.no_grad()`** disables gradient tracking. We are not computing any loss or doing any backward pass, so there is no need to store the computation graph. This saves substantial memory.

6.2 Greedy Decoding

The simplest strategy: always pick the token with the highest probability.

Conceptual — illustrating the idea

```
next_token = logits.argmax(dim=-1)
```

This is deterministic — the same input always produces the same output. Greedy decoding is what the Annotated Transformer's `greedy_decode()` function uses for machine translation, where there is typically one correct translation and you want the most likely one.

For open-ended text generation, greedy decoding produces repetitive, generic text. The model always picks the "safe" choice — the most common next character given the context. This leads to loops ("the the the...") and bland output. The model has learned a rich probability distribution over possible continuations, but greedy decoding collapses that distribution to a single point.

6.3 Temperature Scaling

Temperature gives us a knob to control the shape of the probability distribution before sampling.

Given logits $z_1, z_2, \ldots, z_K$ (one per vocabulary token), divide by temperature T before applying softmax:

$$P(\text{token}_i) = \frac{\exp(z_i/T)}{\sum_{j=1}^{K} \exp(z_j/T)}$$

The effect depends on T:

$T = 1.0$ (default): The distribution is unchanged. This is what the model was trained to produce.

$T \to 0$: The distribution becomes sharper. Dividing by a small T amplifies the differences between logits. If the highest logit is 5.0 and the second-highest is 4.5, dividing by $T = 0.1$ makes them 50.0 and 45.0 — the gap from 0.5 to 5.0. After softmax, the highest-logit token dominates exponentially. In the limit, this approaches greedy decoding (argmax).

$T > 1.0$: The distribution becomes flatter. Dividing by a large T compresses the differences between logits. All tokens become more equally likely. The model samples more randomly, producing more varied but less coherent text.

$T < 1.0$ (but above 0): The distribution becomes sharper than the model's default. High-probability tokens become even more dominant, low-probability tokens become even less likely. This produces more conservative, predictable text.

Conceptual — illustrating the idea

From `BabyGPT.generate` in `babygpt/model.py`.

```
# In our generate() method:
logits = logits[:, -1, :] / temperature  # scale before softmax
probs = F.softmax(logits, dim=-1)
next_token = torch.multinomial(probs, num_samples=1)
```

Practical range: 0.5 to 1.2 covers most use cases. Below 0.5, output is nearly deterministic. Above 1.2, the model starts making frequent errors.

Tip: Why is it called "temperature"? The name comes from statistical mechanics. The Boltzmann distribution gives the probability of a physical system occupying a state with energy E_i at temperature T: $P(\text{state}_i) \propto \exp(-E_i/kT)$. Our softmax-with-temperature has the same functional form (the sign is flipped because higher logits are *better*, whereas lower energies are more favorable). The analogy maps exactly: high physical temperature means particles spread across many energy states (more disorder); high softmax temperature means probability spreads across many tokens (more randomness). As $T \to 0$, a physical system freezes into its lowest-energy ground state; our model freezes into argmax (greedy decoding). "Turn up the temperature" means the same thing in both worlds: more thermal energy, more randomness.

6.4 Top-k Sampling

Temperature controls the shape of the distribution, but it does not eliminate the long tail. Even at moderate temperatures, the model occasionally samples from tokens with very low probability — characters that make no sense in context. These rare samples produce garbage characters that derail the text.

Top-k sampling fixes this by restricting the sample to the k most probable tokens. All other tokens are set to $-\infty$ before softmax, which gives them zero probability.

Conceptual — illustrating the idea

From `BabyGPT.generate` in `babygpt/model.py`.

```
if top_k is not None:
    top_values, _ = torch.topk(logits, min(top_k, logits.size(-1)))
    logits[logits < top_values[:, [-1]]] = float("-inf")

probs = F.softmax(logits, dim=-1)
next_token = torch.multinomial(probs, num_samples=1)
```

Step by step:

1. `torch.topk(logits, k)` returns the k largest logit values (and their indices).
2. `top_values[:, [-1]]` is the smallest value among the top k — the threshold.
3. Every logit below this threshold is set to $-\infty$.
4. Softmax converts $-\infty$ to probability 0. Only the top k tokens have nonzero probability.
5. `torch.multinomial` samples from this truncated distribution.

Typical values: $k = 40$ to $k = 200$.

The trade-off: too small a k restricts the model to only the most obvious continuations, producing repetitive text (similar to low temperature or greedy decoding). Too large a k lets through low-probability tokens that introduce noise. For a character-level model with a 65-token vocabulary, even $k = 40$ leaves most of the vocabulary available — top-k is more impactful with larger vocabularies (e.g., BPE with 50K tokens, where the tail contains many irrelevant subwords).

Temperature and top-k work well together: temperature controls the relative probabilities among the surviving tokens, while top-k eliminates the tail.

6.5 Top-p (Nucleus) Sampling – Concept

Top-k uses a fixed number of tokens regardless of the distribution shape. This can be suboptimal: when the model is very confident (one token has 95% probability), $k = 50$ includes 49 near-zero-probability tokens unnecessarily. When the model is uncertain (probability spread across many tokens), $k = 50$ might cut off plausible continuations.

Top-p (nucleus) sampling (Holtzman et al., 2019) adapts to the distribution. Instead of keeping a fixed count of tokens, it keeps the smallest set of tokens whose cumulative probability exceeds a threshold p:

1. Sort tokens by probability, highest first.
2. Compute cumulative probabilities: $c_1 = p_1$, $c_2 = p_1 + p_2$, etc.
3. Include tokens until $c_i \geq p$. Set all remaining tokens to $-\infty$.

For confident predictions (one token at 95%), the nucleus contains just 1–2 tokens. For uncertain predictions (many tokens at 2–5% each), the nucleus might contain 20–30 tokens. The sampling adapts automatically to the model's confidence.

Typical values: $p = 0.9$ to $p = 0.95$.

We do not implement top-p in our code — top-k is sufficient for a character-level model with a 65-token vocabulary. But top-p (often combined with temperature) is the standard method used in production language models (GPT-4, Claude, etc.).

6.6 The Generate Function

First, add the `generate()` method to the `BabyGPT` class in `babygpt/model.py`. This method implements the full autoregressive loop from Sections 1–4: forward pass, temperature scaling, top-k filtering, sampling, and appending.

File: babygpt/model.py -- add to BabyGPT class, after forward

```
@torch.no_grad()
def generate(
    self,
    tokens: torch.Tensor,
    max_new_tokens: int,
    temperature: float = 1.0,
    top_k: int | None = None,
) -> torch.Tensor:
    """Autoregressive token generation.

    Loop: forward pass -> logits / temperature -> top-k filter
          -> softmax -> multinomial sample -> append token.
    Crops to context_size when sequence exceeds context window.
```

```
    """
    for _ in range(max_new_tokens):
        tokens_cropped = (
            tokens if tokens.size(1) <= self.config.context_size
            else tokens[:, -self.config.context_size:]
        )

        logits, _ = self(tokens_cropped)
        logits = logits[:, -1, :] / temperature  # (batch, vocab_size)

        if top_k is not None:
            top_values, _ = torch.topk(logits, min(top_k,
               logits.size(-1)))
            logits[logits < top_values[:, [-1]]] = float("-inf")

        probs = F.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)  # (batch,
           1)
        tokens = torch.cat([tokens, next_token], dim=1)

    return tokens
```

Next, we need a CLI script to load a trained checkpoint, encode a prompt, call `model.generate()`, and print the result.

6.6.1 Creating the Generation Script

Create `babygpt/generate.py` — the CLI wrapper that loads a checkpoint, encodes the prompt, calls `model.generate()`, and prints the result:

File: babygpt/generate.py

```
"""Generate text from a trained BabyGPT checkpoint."""

import argparse
from pathlib import Path

import torch

from .config import GPTConfig
from .model import BabyGPT
from .tokenizer import CharTokenizer

CHECKPOINT_DIR = Path(__file__).resolve().parent.parent / "checkpoints"


def load_model(
    checkpoint_path: Path,
    device: torch.device,
) -> tuple[BabyGPT, CharTokenizer]:
    """Load a model and tokenizer from a checkpoint."""
    checkpoint = torch.load(checkpoint_path, map_location=device,
       weights_only=False)

    config = checkpoint["config"]
    model = BabyGPT(config)
    model.load_state_dict(checkpoint["model"])
```

```
    model.to(device)
    model.eval()

    # Reconstruct tokenizer from saved vocab
    vocab = checkpoint["vocab"]
    tokenizer = CharTokenizer.__new__(CharTokenizer)
    tokenizer.char_to_index = vocab
    tokenizer.index_to_char = {i: ch for ch, i in vocab.items()}

    print(f"Loaded model from {checkpoint_path}")
    print(f"  step={checkpoint.get('step', '?')},
        val_loss={checkpoint.get('val_loss', '?'):.4f}")

    return model, tokenizer


def generate(
    model: BabyGPT,
    tokenizer: CharTokenizer,
    prompt: str,
    max_new_tokens: int = 500,
    temperature: float = 1.0,
    top_k: int | None = None,
    device: torch.device = torch.device("cpu"),
) -> str:
    """Generate text given a prompt string."""
    token_ids = tokenizer.encode(prompt)
    tokens = torch.tensor([token_ids], dtype=torch.long, device=device)

    output = model.generate(tokens, max_new_tokens,
        temperature=temperature, top_k=top_k)

    return tokenizer.decode(output[0].tolist())


def main() -> None:
    parser = argparse.ArgumentParser(description="Generate text with
        BabyGPT")
    parser.add_argument("--prompt", type=str, default="\n",
        help="Starting text")
    parser.add_argument("--max-tokens", type=int, default=500,
        help="Tokens to generate")
    parser.add_argument("--temperature", type=float, default=0.8,
        help="Sampling temperature")
    parser.add_argument("--top-k", type=int, default=None, help="Top-k
        sampling")
    parser.add_argument(
        "--checkpoint",
        type=str,
        default=str(CHECKPOINT_DIR / "best.pt"),
        help="Path to model checkpoint",
    )
    args = parser.parse_args()

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model, tokenizer = load_model(Path(args.checkpoint), device)

    text = generate(
```

```
        model,
        tokenizer,
        prompt=args.prompt,
        max_new_tokens=args.max_tokens,
        temperature=args.temperature,
        top_k=args.top_k,
        device=device,
    )

    print("\n" + "=" * 60)
    print(text)
    print("=" * 60)


if __name__ == "__main__":
    main()
```

Shell — run in your terminal

```
uv run python -m babygpt.generate --prompt "ROMEO:" --temperature 0.8
```

Checkpoint: You should see several lines of Shakespeare-like text with correct dialogue formatting (character names in caps followed by colons) and quasi-Elizabethan vocabulary. Try different temperatures: 0.5 for conservative output, 1.2 for more creative.

6.7 Example Outputs

The quality of generated text varies significantly with temperature. Here is what to expect from a trained BabyGPT at different settings:

Temperature 0.5 (conservative): Correct spelling and formatting, but repetitive. The same phrases recur.

Temperature 0.8 (recommended): Good balance of coherence and variety. The default in `babygpt/generate.py`.

Temperature 1.2 (creative): More surprising choices, but more spelling errors and occasional gibberish.

Important: this is a **character-level** model trained on 1MB of text. The model learns:

- Character-level patterns: common letter sequences, word boundaries, capitalization
- Formatting conventions: character names in uppercase followed by colons, verse-like line breaks, act/scene headers
- Pseudo-Elizabethan spelling: sequences like "thou", "hath", "thy" appear frequently because they are statistically common

The model does **not** learn:

- Semantic coherence: sentences do not form a logical narrative
- Plot or character consistency: the model does not track who is speaking or what they are talking about

- Factual content: the model cannot reference actual Shakespeare plots

This is not a limitation of the architecture — it is a limitation of scale. The same architecture, with more parameters, subword tokenization, and more data, produces GPT-3.

6.8 What Would Make It Better

Our BabyGPT is a minimal working example. Each of the following changes would produce measurable improvements, and they are roughly ordered by impact:

More data. TinyShakespeare is 1MB. GPT-2 was trained on 40GB (WebText). GPT-3 on 570GB (filtered Common Crawl + books + Wikipedia). More data means more patterns to learn and less overfitting. This is the single most impactful change.

Subword tokenization (BPE). Character-level tokenization means the model must use its context window to learn that "t", "h", "e" form the word "the." With BPE, "the" is a single token, and the model can use its context window for higher-level patterns. A 256-token context window covers roughly 1000 characters with BPE versus 256 characters with character-level tokenization.

Larger model. More layers and more embedding dimensions increase the model's capacity to represent patterns. Scaling laws (Kaplan et al., 2020) show that loss decreases as a power law with model size, training data, and compute — and these three factors are roughly interchangeable. For a fixed compute budget, there is an optimal balance between model size and data size (Hoffmann et al., 2022 — the Chinchilla scaling laws).

More compute (longer training). 5000 iterations over 1M tokens is fast but leaves room for further optimization. Longer training with more data would continue to reduce loss.

These are exactly the insights behind the GPT scaling papers. The architecture we built is essentially the same architecture (up to minor details like attention implementations and normalization choices) used in models with 100 billion+ parameters. The difference is entirely in scale: more parameters, more data, more compute.

This concludes the six-chapter series. The full pipeline — from the attention mechanism to a trained, generating language model — is implemented in `babygpt/`.

6.9 Project Summary

At this point your project directory should contain:

```
learn-attn-impl/
  pyproject.toml, ... uv files
  babygpt/
    __init__.py
    config.py       (Chapter~4)
    model.py        (Chapter~4)
    tokenizer.py    (Chapter~5)
    dataset.py      (Chapter~5)
    train.py        (Chapter~5)
    generate.py     (Chapter~6)
  data/
    tinyshakespeare.txt
  checkpoints/
```

```
best.pt          (saved by babygpt/train.py)
```

The model is 10.7M parameters, trained on 1MB of Shakespeare, generating character-level text. The same architecture — with more parameters, subword tokenization, and more data — is what powers GPT-2, GPT-3, and their successors.